

Optimisation and Deep Learning

(CT118-3-3-ODL)

Individual Assignment

Student Name : **Lim Beng Rhui**

TP Number : **TP068495**

Intake Code : **APD3F2601CS(DA)**

Lecturer Name : **Dr. Wong Kuan Wai**

Hand out Date : **10 July 2026**

Hand in Date : **18 September 2026**

Table of Contents

Introduction ····· 4
Literature Review ····· 5
Dataset and Algorithm Selection ····· 8
Dataset Selection ····· 8
Algorithm Selection ····· 9
Data Preparation ····· 10
Setup and Configuration ····· 10
Dataset Loading ····· 11
Class Distribution ····· 13
Missing and Invalid Values ····· 14
Visual Inspection on Positive and Negative Samples ····· 15
Tissue Content Verification using Saturation ····· 16
Duplicate Detection and Cross-Split Leakage ····· 18
Stratified Sampling of the Training Split ····· 19
Computing the Normalisation Statistics ····· 20
Colour Deconvolution and Data Augmentation ····· 21
Final Verification of the Prepared Dataset ····· 25
Model Building ····· 27
Environment and Preprocessing Setup ····· 27
Model Architecture ····· 29
Vanilla Convolutional Neural Network (CNN) ····· 29
CNN-Bidirectional Long Short-Term Memory (CNN-BiLSTM) ····· 31
Model Training ····· 35
Training Configuration ····· 35
Training Baseline CNN and Hybrid CNN-BiLSTM ····· 39
Comparison of Validation Results and Learning Curves of Models ····· 40
Model Tuning ····· 41
Tuning Configuration ····· 41
Search Space ····· 42
Random Search ····· 44
Best Hyperparameter Configurations and Influence of Hyperparameters ····· 45
Retraining of Optimised Models ····· 47

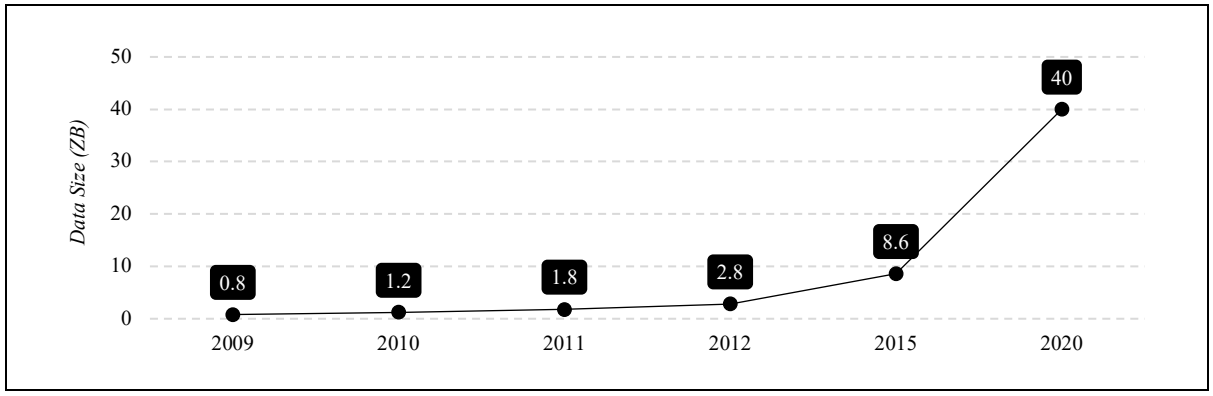
Model Evaluation 48
Determining the Decision Threshold 48
Evaluation Metrics 49
Performance Comparison of Baseline and Optimised Models 50
Champion Model Selection 53
Final Evaluation of Champion Model on Test Set 54
Conclusion 57
Limitations 58
Recommendations 59
References 60

Introduction

In recent years, technologies like artificial intelligence and big data have contributed significantly across different industries. Businesses are actively applying data analytics and visualisation tools to develop targeted marketing strategies and increase their revenues, while researchers also integrate data science methodologies into their experiments to uncover hidden patterns within their data and obtain scientific breakthroughs (Udeh et al., 2024). Among these industries, the healthcare industry emerges as one of the fields with the greatest potential to benefit from data analytics. As shown in Figure 1, Bojnordi et al. (2025) outlined that the volume of healthcare data collected grows exponentially across recent years, yet Cao and Ma (2025) unveils that more than 95% of these data are unused due to several challenges, such as the lack of infrastructure to integrate healthcare data analytics into clinical workflows. The absence of data-integrated tools impacts doctors' efficiency when performing diagnosis on critical illnesses, such as breast cancer that becomes difficult to be cured once it is metastasised (Burguin et al., 2021). According to the World Health Organization (2026), approximately 2.4 million women were diagnosed with breast cancer in 2024 alone, resulting in roughly 694,000 deaths across the globe. Therefore, this project applies deep learning modelling techniques to classify images of lymph node tissue and determine if the breast cancer cells have metastasised to the lymph nodes, aiming to assist doctors to evaluate the stages of breast cancer more effectively to provide accurate and timely treatment decisions to patients, thereby increasing their survival rates while reducing the need of aggressive treatments (Edwards et al., 2025).

Figure 1

Exponential Growth of Healthcare Data Volume



Note. From "Identifying, ranking and analyzing obstacles to big data analytics implementation in the healthcare industry using an ISM approach," by A. J. J. Bojnordi, M. Z. Nezhad, R. Bagheri, M. Bazrafshan, and B. Sohrabi, 2025, *Discover Health Systems*, *4*(32).

Literature Review

In 2022, breast cancer was listed as the most prominent cause of cancer death among women globally, according to the study from Bray et al. (2024). This is because the symptoms for breast cancer are subtle at its early stage, where patients may experience lumps or inflammation at their breast or axilla (Sharma & Saripilli, 2025) that are easily unnoticed without routine screening tests. Especially in low and middle-income countries that lack sufficient medical fundings and resources to perform screening programmes for their citizens, this often leads to significant delays in diagnosis and late-stage detection, in which cancer cells have already spread to other parts of the body, such as the lymph nodes, and resulting in poorer prognosis (Salama et al., 2026). Therefore, it is necessary for oncologists to opt for solutions that are able to accurately evaluate patients' conditions, so that the subsequent treatments to tackle breast cancer can be determined correctly to avoid undertreatment or overtreatment from taking place on patients.

Nonetheless, before diving deep into existing solutions, understanding the underlying mechanisms of breast cancer is crucial to establish a therapeutical foundation for breast cancer treatments. When breast cancer cells mutate and multiply uncontrollably, they breach the ductal wall and invade surrounding tissues, where cancer cells that enter the lymphatic vessels are transported to the axillary lymph nodes, while those that enter the blood vessels are transmitted around the body via the blood circulation system, thereby causing distant metastases on organs like bones and lungs (Liu et al., 2021). While techniques like liquid biopsy are designed to detect cancer cells that circulate within the human body with minimal invasive (Connal et al., 2023), Kotsifaki et al. (2026) argued that liquid biopsy encounters several limitations, such as suffering from biological background noise that impacts analysis accuracy and performance, as well as requiring highly complex infrastructure and technological resources to support its adoption and implementation, making it appropriate for breast cancer detection and monitoring rather than anatomical staging. On the other hand, sentinel lymph node biopsy (SLNB) is the golden standard for breast cancer staging due to its safe and effective implementation under environments that are resource-constrained (Piso et al., 2026). This also aligns with the study from Feng et al. (2026) who pointed out that, compared to the traditional axillary lymph node dissection (ALND), SLNB is able to reduce the rate of morbidity among patients while providing pathological information, such as the size of metastatic foci and the presence of extracapsular extension (ECE), that allows oncologists to determine the extent of cancer cell spread in a more accurate manner to assist them in outlining follow-up treatments for patients.

Typically, when patients are diagnosed with breast cancer and undergo surgeries, aside from performing lumpectomy or mastectomy to remove the primary breast tumour, surgical oncologists will also perform SLNB by extracting the sentinel lymph nodes from the axilla (Aydin et al., 2026). Once the extracted nodes are sent to the pathology lab for microscopic analysis, they will be sectioned and stained with haematoxylin and eosin (H&E) by laboratory staff to ensure that the tissues of the nodes are visible to determine if metastatic deposits are present, thereby informing further treatment decisions (Barnachea & Lee, 2026). However, after converting tissue slides into digitalised whole-slide images (WSI) with billions of pixels, scrutinising these images for the presence of metastatic deposits is a demanding task, where pathologists often have to scan through large areas of healthy tissues to locate the tiny clusters of cancer cells, then integrate the findings obtained from different slides to synthesise their interpretation (Koyun et al., 2026). Especially when performing the task under time and workload pressure, pathologists, who have to continuously provide visual attention across a prolonged timeframe, often suffer degrading attention spans that may lead to misinterpretation (Kim et al., 2026), yielding severe consequences to patients, as a missed micrometastasis results in patients unable to receive their adjuvant therapies, while false positives cause patients to undergo prolonged yet unnecessary treatment that may jeopardise their safety and health.

Hence, researchers have attempted to minimise human diagnosis errors by proposing multiple automated methods to analyse sentinel lymph nodes. Early pioneers like Weaver et al. (2003) have utilised an image analysis system to differentiate colour-coded cancer cells using rule-based techniques, while recent studies like Yang et al. (2025) applied logistic regression and tree-based models trained on clinical data to predict the status of sentinel lymph nodes preoperatively. Nonetheless, Balkenende et al. (2022) argued that traditional computer-aided detection/diagnosis (CAD) approaches do not contribute to the improvement of diagnostic accuracy, aligning with the findings from Leo et al. (2018) who discovered that features that are manually engineered may experience instability when switching between different experiment sites, reflecting the poor generalisability of models that rely on these attributes. To tackle such issues, researchers like Windsor et al. (2023) and Srinidhi et al. (2021) have recommended the implementation of deep learning models that automatically learn features directly from image data with minimal human interference, outperforming traditional machine learning models. Several state-of-the-art deep learning models applied for sentinel lymph node classification are outlined in Table 1, where it is observed that convolutional neural network (CNN) is employed by most studies, thus informing the methodological direction of this project.

Table 1

Comparison of Related Works on Lymph Node Metastasis Classification Using Deep Learning

Works	Dataset Used	Model Used	Purpose	Metrics	Findings
Wang et al. (2021)	Rectified PatchCamelyon (RPCam)	Boosted EfficientNet-B3 (*Boosted CNN*)	Binary classification of lymph node metastasis	• Accuracy : 97.96% • ROC-AUC : 0.9968 • F1-Score : 0.9747	• Random centre cropping (RCC) technique increases ROC-AUC from 0.9924 to 0.9954 • Combining attention and feature fusion mechanisms yields highest performance gain • Boosted EfficientNet-B3 outperforms on all metrics with fewer parameters
Bütün et al. (2023)	PatchCamelyon from Kaggle (K-PCam)	ResNet-34/50/101 (*CNN*)	Binary classification of lymph node metastasis	• Accuracy : 98.60% • ROC-AUC : 0.9853	• Integrate ResNet with 1cycle policy yields best performance without model redesign • Applying transfer learning yields lower results compared to without transfer learning
Chang et al. (2025)	954 sentinel lymph node frozen sections (*Self-gathered*)	Convolutional neural network (*CNN*)	Binary classification of metastatic lesions in sentinel lymph nodes	• Sensitivity : 98.8% • Specificity : 62.1%	• Training on 72 samples over 954 cases is able to achieve high recall of 98.8% • Designed to accept high false positive rates to ensure that pathologists will not miss out metastasis-positive cases during assessment
Tauqeer et al. (2025)	CAMELYON grand challenge dataset (CAMELYON16, CAMELYON17)	Hybrid framework: • MaxViT for nuclei feature extraction • CTransPath for tissue feature extraction • Selective attention mechanism for classification	Binary classification of • Patch-level tumour detection • Slide-level metastasis in sentinel lymph nodes	<u>CAMELYON16 Dataset</u> Patch-level tumour • Sensitivity : 96.2% • Precision : 95.3% • F1-Score : 0.957 Slide-level metastasis • ROC-AUC : 0.96	• Nuclei-level features and selective neighbourhood attention mechanism are able to improve prediction performance • Consistent generalisability across different datasets despite slightly poorer performance on the dataset CAMELYON17

Dataset and Algorithm Selection

Dataset Selection

With the goal of automating breast cancer staging to streamline pathological workflows while minimising human error and diagnostic misinterpretation, this project uses the PatchCamelyon (PCam) dataset by Veeling et al. (2018) to train and evaluate multiple deep learning models that determine the presence of metastasis in sentinel lymph nodes. Although similar datasets, such as the CAMELYON16 dataset, are available, they often provide raw whole-slide images (WSI) that not only require researchers to install specialised libraries like OpenSlide to render the raw slide files, but also consume a massive amount of computational power and resources throughout the preprocessing and training stages (Ashraf et al., 2022). Therefore, the PCam dataset, which is derived by slicing the WSI of sentinel lymph nodes from the CAMELYON16 dataset into 327,680 patches of images with size of 96×96 pixels, is chosen for this project, not only due to its clinical relevance, but also due to its computational feasibility as the dataset allows deep learning models to be trained on devices with limited computational power while achieving performance on par with models trained using the CAMELYON16 dataset.

```
Link to Dataset: https://patchcamelyon.grand-challenge.org
```

The PCam dataset consists of patches of sentinel lymph node tissues stained in haematoxylin and eosin (H&E), with positive labels representing that tumour appears at the centre 32×32 pixel region. This indicates that tumours that appear outside the region are not flagged, which, despite the mismatch between input and label, is a deliberate decision to allow convolutional models to be applied without zero-padding. The dataset is further split into three sets without any repetition of WSI, where the training set contains 262,144 images, while both the validation and testing sets contain 32,768 images, each with an approximately equal number of positive and negative cases. Although the PCam splits are constructed with balanced classes, due to the rare occurrence of metastasis in sentinel lymph nodes (van Dooijeweert et al., 2024), the metrics obtained from the PCam dataset may not fully align with results from real-world deployment, thereby requiring careful interpretation before integrating trained models into clinical workflows. Additionally, the patches are collected from only two centres, which creates a risk for models to learn the staining characteristics of patches instead of tissue morphology (Tellez et al., 2019) and negatively impacts their generalisability. Yet, the PCam dataset remains as the choice for this project as it is an established benchmark for sentinel lymph node classification, where several of these limitations will be addressed in subsequent stages.

Algorithm Selection

Based on the comparison of similar studies in Table 1, it is observed that most literature adapts convolutional-based architectures when performing classification tasks to predict the presence of metastasis in sentinel lymph nodes. Therefore, this project proposes the implementation of two convolutional-based deep learning models:

- **Vanilla Convolutional Neural Network (CNN)**

  The vanilla convolutional neural network (CNN) is selected to establish a baseline for model comparison. As the classification task requires image patches of sentinel lymph nodes to be analysed, the underlying mechanisms of CNN, such as hierarchical feature extraction and shared convolutional filters, allow the CNN model to learn image features automatically without causing parameter explosion (Alzubaidi et al., 2021). Furthermore, the nature of these image patches aligns with the three inductive biases of CNN, including (1) locality, as diagnostic cues are often located within neighbouring pixel regions, (2) translation equivariance, which suits the fact where metastatic cells may appear anywhere within the label region of a patch, and (3) compositionality, as the tissue architecture is built from smaller components like nuclei and cells (Bronstein et al., 2021). This reduces the volume of data required for CNN to achieve good generalisation (Dosovitskiy et al., 2021), making the vanilla CNN suitable to be the baseline model as it can be trained efficiently without requiring excessive computational resources, thereby forming the foundation for the development of the hybrid CNN-BiLSTM model.

- **CNN-Bidirectional Long Short-Term Memory (CNN-BiLSTM)**

  As an extension of the baseline CNN model, a hybrid CNN-BiLSTM is proposed as the alternative model for this project. Although CNN models are able to effectively extract image features from local pixel regions, the convolutional operations process each local neighbourhood one at a time, while subsequent pooling operations aggregate features without considering the relationship between different regions of a patch (Wang et al., 2018). Therefore, an additional bidirectional long short-term memory (BiLSTM) layer is applied to complement the existing convolutional layers by allowing the model to treat each region sequentially while permitting contextual information to be transmitted in both directions across the different regions of a patch. This aligns with Srikantamurthy et al. (2023) who reported that the hybrid CNN-LSTM architecture is able to outperform established baseline convolutional networks on breast histopathology datasets.

Data Preparation

Setup and Configuration

Before performing data cleaning and exploratory data analysis (EDA), all libraries required for the project are imported in Figure 2, together with the declaration of directories and dataset constants. Two additional functions are also implemented, where `set_seed()` sets a random seed to ensure that results are reproducible, while `get_device()` helps to determine the hardware available to `PyTorch` to speed up subsequent EDA and model training tasks.

Figure 2

Setup and Configuration of the Preprocessing Notebook

```
# Standard library
import os, random, hashlib, warnings
from pathlib import Path

# Third-party
import numpy as np, pandas as pd, torch, h5py
import matplotlib.pyplot as plt
from torchvision.transforms.v2.functional import rgb_to_grayscale

# Suppress warnings
warnings.filterwarnings("ignore", category=UserWarning)

# Directories for the project
PROJECT_ROOT         = Path("..")
RAW_DIRECTORY        = PROJECT_ROOT  / "data" / "raw"
PROCESSED_DIRECTORY  = PROJECT_ROOT  / "data" / "processed"
TRAINING_DIRECTORY   = RAW_DIRECTORY / "training"
VALIDATION_DIRECTORY = RAW_DIRECTORY / "validation"
TESTING_DIRECTORY    = RAW_DIRECTORY / "testing"
H5_FILES = {
    "train_x": TRAINING_DIRECTORY   / "camelyonpatch_level_2_split_train_x.h5",
    "train_y": TRAINING_DIRECTORY   / "camelyonpatch_level_2_split_train_y.h5",
    "valid_x": VALIDATION_DIRECTORY / "camelyonpatch_level_2_split_valid_x.h5",
    "valid_y": VALIDATION_DIRECTORY / "camelyonpatch_level_2_split_valid_y.h5",
    "test_x": TESTING_DIRECTORY     / "camelyonpatch_level_2_split_test_x.h5",
    "test_y": TESTING_DIRECTORY     / "camelyonpatch_level_2_split_test_y.h5",
}

# Constants of the PCam dataset
# Obtained from the official GitHub site: https://github.com/basveeling/pcam
IMG_SIZE      = 96        # The size of image is 96x96 pixels
CENTRE_REGION = 32        # Label is defined by the center 32x32 region
N_TRAIN_FULL  = 262_144   # Number of training samples
N_VALID_FULL  = 32_768    # Number of validation samples
N_TEST_FULL   = 32_768    # Number of testing samples

# A function to set a seed to ensure reproducibility
SEED = 42
def set_seed(seed: int = SEED) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
    os.environ["PYTHONHASHSEED"] = str(seed)

# A function to determine the notebook or device to run the model
def get_device() -> torch.device:
    if torch.backends.mps.is_available():        # Apple Silicon's GPU
        return torch.device("mps")
    if torch.cuda.is_available():                # NVIDIA GPU
        return torch.device("cuda")
    return torch.device("cpu")                   # Fallback CPU - Slowest performance

# Set a seed for reproducibility
set_seed(SEED)

# Identify the device to run the notebook
DEVICE = get_device()
```

Dataset Loading

With all libraries and directories configured, the PCam dataset is downloaded from the Grand Challenge (2026) platform, and the respective training, validation and testing sets are manually loaded into the `data/raw` directory, as shown in Figure 3. These datasets are distributed in the HDF5 (`.h5`) format that allows the image patches to be accessed directly without having to load the entire dataset file into memory (Butt et al., 2026), hence reducing the memory consumption required when performing data preprocessing steps. Using the code snippet in Figure 4, the file size and the number of records in each dataset are retrieved, not only to ensure that each dataset file is valid and accessible, but also to reconcile with the counts from the official PCam GitHub repository to make sure there are no missing rows across each dataset. The content of the dataset is also previewed by extracting the first five image patches in the training set. As shown in Figure 5, patches that are labelled with `label = 0` do not contain any metastatic tissue in their centre 32×32 region, while patches with `label = 1` indicate the opposite. With these results, it is confirmed that the PCam dataset loaded into the project is complete and accessible, allowing the project to proceed with subsequent preprocessing tasks.

Figure 3

Project Directory Structure After Loading the PCam Dataset

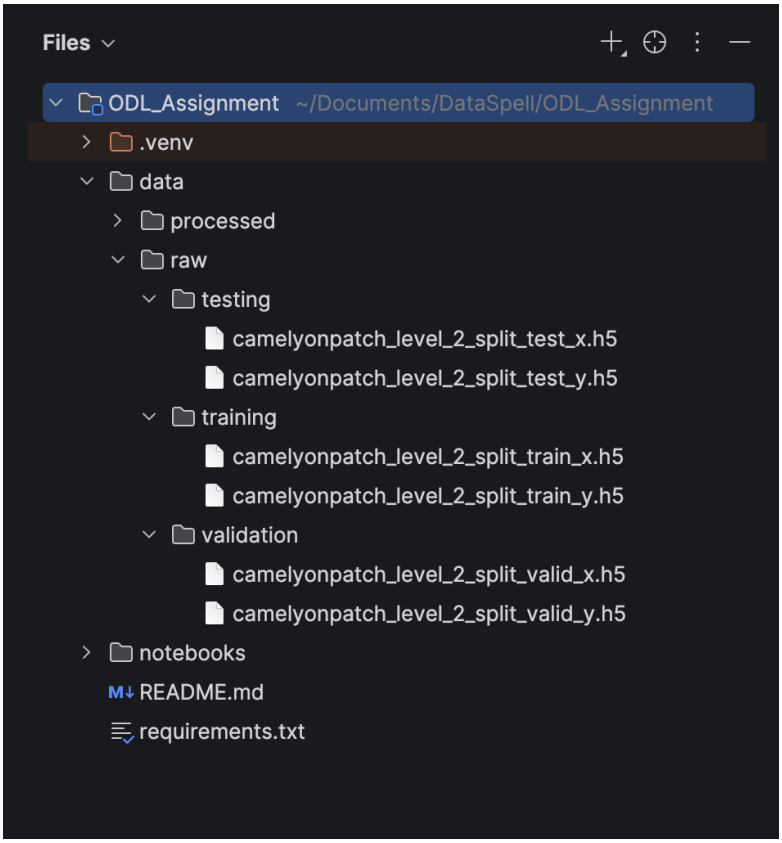


Figure 4

Retrieving the File Size and Row Count of Each Dataset

```
# The names of each dataset split
SPLITS = ("train", "valid", "test")

# The mapping of each split to their number of samples
EXPECTED_N = {"train": N_TRAIN_FULL, "valid": N_VALID_FULL, "test": N_TEST_FULL}

# Retrieve the details of each dataset
rows = []
for name, path in H5_FILES.items():
    split, key = name.split("_")          # Split the name into "train", "x"
    with h5py.File(path, "r") as file:    # Extract the information from the HDF5 file
        rows.append({
            "file": path.name,
            "size_mb": round(path.stat().st_size / 1e6, 2),
            "rows": file[key].shape[0],
            "shape": file[key].shape,
            "expected": EXPECTED_N[split],
        })

# Convert the extracted details into table and display
files = pd.DataFrame(rows)
display(files)
```

	file	size_mb	rows	shape	expected
0	camelyonpatch_level_2_split_train_x.h5	7248.30	262144	(262144, 96, 96, 3)	262144
1	camelyonpatch_level_2_split_train_y.h5	0.26	262144	(262144, 1, 1, 1)	262144
2	camelyonpatch_level_2_split_valid_x.h5	906.04	32768	(32768, 96, 96, 3)	32768
3	camelyonpatch_level_2_split_valid_y.h5	0.03	32768	(32768, 1, 1, 1)	32768
4	camelyonpatch_level_2_split_test_x.h5	906.04	32768	(32768, 96, 96, 3)	32768
5	camelyonpatch_level_2_split_test_y.h5	0.03	32768	(32768, 1, 1, 1)	32768

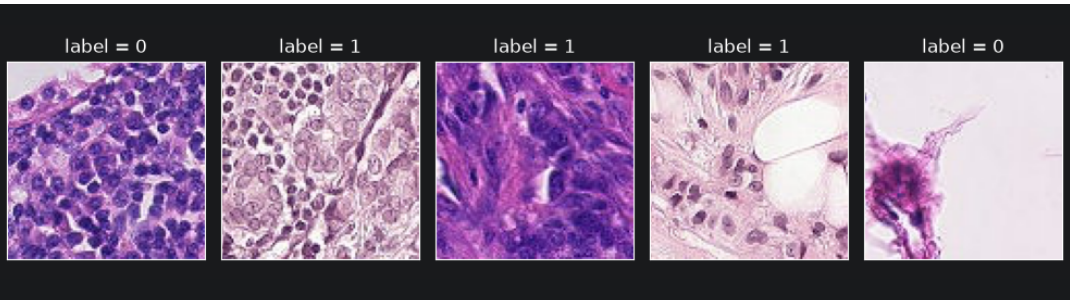
Figure 5

Previewing the First Five Training Patches and Their Labels

```
# Extract the first five images from the training dataset
with h5py.File(H5_FILES["train_x"], "r") as file:
    images = file["x"][:5]

# Extract the first five labels from the training dataset
with h5py.File(H5_FILES["train_y"], "r") as file:
    labels = file["y"][:5].reshape(-1)

# Display the first five patches with their labels
fig, axes = plt.subplots(1, 5, figsize=(10, 2.4))
for ax, image, label in zip(axes, images, labels):
    ax.imshow(image)
    ax.set_title(f"label = {label}")
    ax.set(xticks=[], yticks=[])
    ax.grid(False)
plt.tight_layout()
plt.show()
```



Class Distribution

After successfully loading the dataset, the distribution of each split is inspected to determine if the PCam dataset suffers from class imbalance, as deep learning models, such as convolutional neural networks (CNN), tend to bias towards the majority class if the issue of imbalance is not addressed (Ghosh et al., 2024). Therefore, to ensure the quality and robustness of the predictive outputs of models, the code snippet in Figure 6 is executed to calculate the number of positive and negative samples in each dataset split. Based on the results, it is shown that all the three splits are evenly distributed, where the training set is perfectly balanced with 131,072 samples in each class, while the validation and testing sets have positive rates of 49.95% and 49.98% respectively, resembling nearly perfect splits. Since all the three splits do not suffer from class imbalance, resampling techniques, such as random undersampling or synthetic oversampling with SMOTE, are not required to balance the ratio of class distribution within the PCam dataset, thereby allowing a wide range of metrics, such as accuracy, F1 score, and ROC-AUC, to be applied and interpreted meaningfully without having to consider the impact of imbalanced class distributions towards these metrics (Aguilar-Ruiz & Michalak, 2024).

Figure 6

Code Snippet for Examining the Class Balance of Each Dataset Split

```
# Read the labels of every split
labels = {}
for split in SPLITS:
    with h5py.File(H5_FILES[f"{split}_y"], "r") as file:
        labels[split] = file["y"][:].reshape(-1).astype(np.uint8)

# Count positive and negative patches per split
balance = pd.DataFrame([
    {
        "split": split,
        "n_images": len(y),
        "n_negative": int((y == 0).sum()),
        "n_positive": int((y == 1).sum()),
        "positive_rate": round(float(y.mean()), 4),
    }
    for split, y in labels.items()
])

# Display the table
display(balance)
```

	split	n_images	n_negative	n_positive	positive_rate
0	train	262144	131072	131072	0.5000
1	valid	32768	16399	16369	0.4995
2	test	32768	16391	16377	0.4998

Missing and Invalid Values

Although the number of image patches matches with the information obtained from the official GitHub repository of the PCam dataset, an additional check is performed to determine if there are any blank images or unexpected data in the dataset, which is carried out by calculating the maximum pixel value for each image, as well as checking the label values in each dataset split. Based on the output in Figure 7, it is discovered that two patches in the training set have a maximum pixel value of 0, indicating that they are blank pictures that do not carry any tissue information and should be excluded when providing inputs to perform model training, so that learning noise can be minimised to enhance model quality. On the other hand, the label files of all the three splits contain only the values 0 and 1, indicating that the labels do not contain any erroneous values and require no additional preprocessing steps to be performed.

Figure 7

Code Snippet for Detecting Blank Patches and Invalid Labels

```
# Check if there is any blank image
for split in SPLITS:

    # Open the file
    blank = 0
    with h5py.File(H5_FILES[f"{split}_x"], "r") as file:

        # Point to the images and calculate the number of patches
        dset = file["x"]
        n = dset.shape[0]

        # Process images in batch of 2048
        for i in range(0, n, 2048):
            block = dset[i:i + 2048]

            # Check if there is images with maximum pixel value of 0
            blank += int((block.reshape(len(block), -1).max(axis=1) == 0).sum())

    # Once done, print the statistics
    print(f"{split:6} {n:>8,} patches | blank: {blank}")

# Check the value of the labels for each split
print()
for split in SPLITS:
    y = labels[split]
    print(split, "| unique labels:", np.unique(y))
```

```
train   262,144 patches | blank: 2
valid    32,768 patches | blank: 0
test     32,768 patches | blank: 0

train | unique labels: [0 1]
valid | unique labels: [0 1]
test | unique labels: [0 1]
```

Visual Inspection on Positive and Negative Samples

To evaluate the difference between the patches showing normal sentinel lymph node tissue and the patches that contain metastatic tissue, the code snippet in Figure 8 is executed to display five positive and five negative samples from the training set, with a green box outlining the central region of the patches. From the output, it is observed that the central regions of both groups contain both pink and purple elements, and the colour intensity varies significantly among patches regardless of their class. This indicates that relying on colour is insufficient to distinguish between positive and negative classes, aligning with Xu et al. (2021) who found that nuclear and tissue features, such as nuclear density and cell orientation, are core factors that indicate the presence of lymph node metastasis. Hence, structural morphology should be prioritised over stained colours during classification, where stain augmentation will be applied as part of preprocessing to prevent models from over-relying on staining intensity when trained.

Figure 8

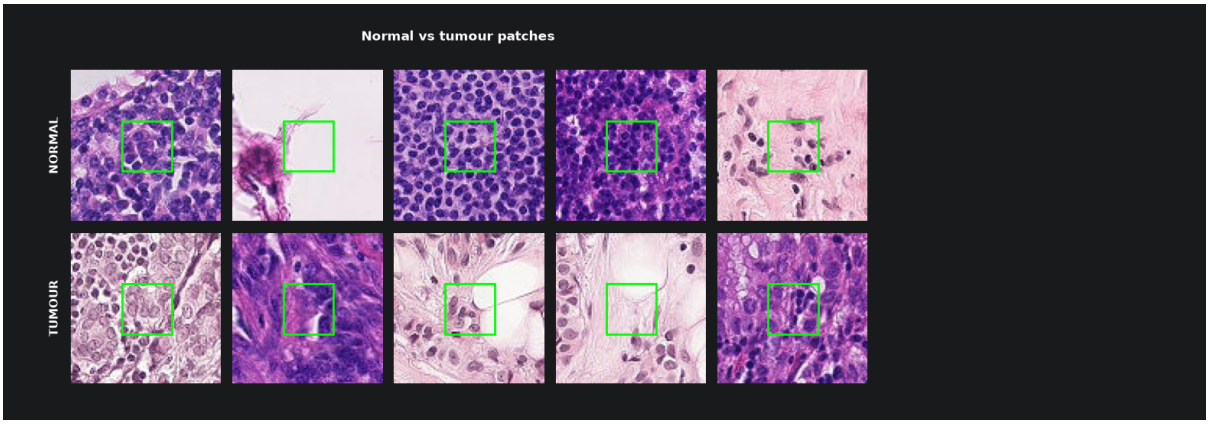
Code Snippet for Displaying Positive and Negative Patches with the Centre Region Outlined

```
# Position of the center region
C0 = (IMG_SIZE - CENTRE_REGION) // 2          # C0 represents the start
C1 = (IMG_SIZE + CENTRE_REGION) // 2          # C1 represents the end

# Extract the IDs of five positive and negative samples from labels
y_train = labels["train"]
normal_idx = np.flatnonzero(y_train == 0)[:5]
tumour_idx = np.flatnonzero(y_train == 1)[:5]

# Extract the pictures based on the label IDs
with h5py.File(H5_FILES["train_x"], "r") as file:
    normal_x = file["x"][normal_idx]
    tumour_x = file["x"][tumour_idx]

# Show each positive and negative samples
fig, axes = plt.subplots(2, 5, figsize=(11, 5))
for row, (patches, name) in enumerate([(normal_x, "NORMAL"), (tumour_x, "TUMOUR")]):
    for col, patch in enumerate(patches):
        ax = axes[row, col]
        ax.imshow(patch)
        ax.add_patch(plt.Rectangle((C0, C0), CENTRE_REGION, CENTRE_REGION, fill=False, color="lime", lw=2))
        ax.set_xticks([])
        ax.set_yticks([])
        ax.grid(False)
        for spine in ax.spines.values():
            spine.set_visible(False)
    axes[row, 0].set_ylabel(name, fontweight="bold", fontsize=11, labelpad=10)
fig.suptitle("Normal vs tumour patches", fontweight="bold")
plt.tight_layout()
plt.show()
```



Tissue Content Verification using Saturation

Aside from the basic checks to identify missing records and labels, an additional verification step is performed to ensure that each patch contains sufficient tissue information to enable deep learning models to learn more effectively. Using the code in Figure 9, the maximum saturation, representing the colourfulness of each patch, is calculated across all the three splits, with the patches containing low tissue information classified using a threshold of 0.07, aligning with the steps performed by Veeling et al. (2018), the creators of the PCam dataset, when cleaning the dataset. Across all the three splits, 622 (approx. 0.19%) patches have a maximum saturation below the threshold, with Figure 10 showing the actual image of 20 of these samples. These patches appear mostly fully white or black, which may be attributed to factors like tissue detection errors or scanning issues (Atallah et al., 2022), indicating that they do not carry sufficient tissue information to support the binary classification task. Although these patches only occupy a small proportion, they will be excluded during the model training phase to ensure that models are able to capture the traits of metastatic lymph node tissue. On the other hand, these patches will be retained in the validation and testing sets, not only to reflect real-world situations where the patches captured might not always be fully visible, but also to ensure that the metrics of the trained models are comparable with published works on the PCam dataset.

Figure 9

Code Snippet for Measuring the Maximum Saturation of Each Patch

```
sat_max, split_of, row_of  = [], [], []

# Loop through each split, open the file, get the image, and process image in batches of 2048
for split in SPLITS:
    with h5py.File(H5_FILES[f"{split}_x"], "r") as file:
        dset = file["x"]
        n = dset.shape[0]
        for i in range(0, n, 2048):
            x = dset[i:i + 2048].astype(np.float32) / 255.0       # Convert the pixel value to the range of [0, 1]
            mx, mn = x.max(axis=3), x.min(axis=3)                 # Saturation: (brightest channel - dimmest channel) / brightest channel
            sat = (mx - mn) / np.maximum(mx, 1e-6)
            sat_max.append(sat.reshape(len(x), -1).max(axis=1))   # Add the maximum saturation to the list
    split_of += [split] * n
    row_of.append(np.arange(n))

# Compile the details into a table
qual = pd.DataFrame({
    "split": split_of,
    "row": np.concatenate(row_of),
    "max_saturation": np.concatenate(sat_max),
})

# Official PCam uses a threshold of 0.07, so those below the threshold are considered low info
SATURATION_THRESHOLD = 0.07
qual["low_tissue"] = qual["max_saturation"] < SATURATION_THRESHOLD
exclude_idx = {s: qual.loc[(qual.split == s) & qual.low_tissue, "row"].to_numpy() for s in SPLITS}

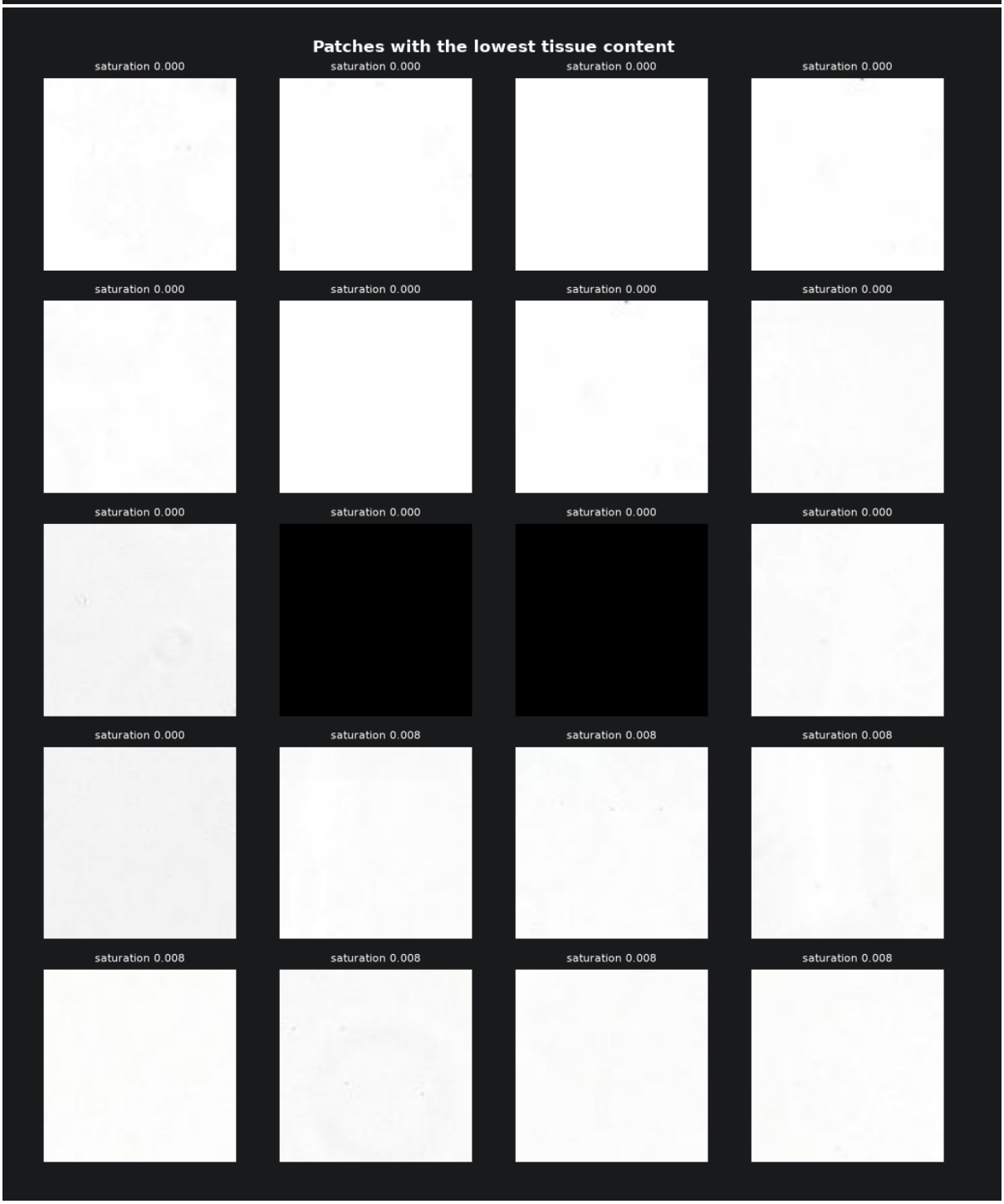
# Show the statistics of the low info tissue in each split
summary = qual.groupby("split", sort=False).agg(n_patches=("split", "size"), low_tissue=("low_tissue", "sum"))
display(summary)
```

split	n_patches	low_tissue
train	262144	505
valid	32768	55
test	32768	62

Figure 10

Code Snippet for Displaying the Twenty Patches with the Lowest Saturation

```
# Show some samples that is below maximum threshold
lowest = qual.nsmallest(20, "max_saturation")
fig, axes = plt.subplots(5, 4, figsize=(10, 12))
for ax, (_, patch) in zip(axes.ravel(), lowest.iterrows()):
    with h5py.File(H5_FILES[f"{patch['split']}_x"], "r") as file:
        ax.imshow(file["x"][int(patch["row"])])
    ax.set_title(f"saturation {patch['max_saturation']:.3f}", fontsize=8)
    ax.axis("off")
fig.suptitle("Patches with the lowest tissue content", fontweight="bold")
plt.tight_layout()
plt.show()
```



Duplicate Detection and Cross-Split Leakage

To prevent duplicated records from affecting the training efficiency of models, a duplicate check is applied to all the three splits of the PCam dataset, as shown in Figure 11. Since the deterministic nature of hashing ensures that two identical image files will always produce the same hash value (Schlett et al., 2024), by using SHA-256, the pixel arrays of each patch are first converted into their corresponding hash values before compared across different image patches and splits to identify the presence of any record duplication or data leakage issues. The results show that each of the split contains a significant number of duplicates, occupying roughly 10% to 16% of the records. Additionally, 33 images from the training set are found to be pixel-identical to patches in the testing set, where this overlap must be addressed to prevent data leakage and ensure that the reported metrics are able to reflect on the true capability of deep learning models. Therefore, these duplicates will be removed from the training set to ensure that the data from the testing split will not be seen by the models, while the validation and testing sets will remain unmodified, not only to maintain class balance, but also to ensure that the metrics obtained are comparable with existing works on the PCam dataset.

Figure 11

Code Snippet for Detecting Duplicate Patches and Cross-Split Leakage

```
# A dictionary to store hash values
hashes = {}

# Loop through each split, open each file, and extract the image column
for split in SPLITS:
    h = []
    with h5py.File(H5_FILES[f"{split}_x"], "r") as file:
        dset = file["x"]
        n = dset.shape[0]
        for i in range(0, n, 2048):              # Convert each image to hash values and add to list
            for image in dset[i:i + 2048]:
                h.append(hashlib.sha256(image.tobytes()).hexdigest())
    hashes[split] = pd.Series(h)

# Calculate and display duplicates in table format
duplicates = pd.DataFrame([
    {"split": split, "n_patches": len(h), "unique": h.nunique(), "duplicates": int(h.duplicated().sum())}
    for split, h in hashes.items()
])
display(duplicates)

# Check whether any image appears in more than one split
train, valid, test = (set(hashes[s]) for s in SPLITS)
print("train & test :", len(train & test))
print("train & valid:", len(train & valid))
print("valid & test :", len(valid & test))

# Retrieve the hash value of the images that are duplicated and leaked
train_hashes = hashes["train"]
repeat_rows = np.flatnonzero(train_hashes.duplicated())
leaked_rows = np.flatnonzero(train_hashes.isin(test))

# Add the rows to be excluded into the exclude_idx list
drop_rows = np.union1d(repeat_rows, leaked_rows)
exclude_idx["train"] = np.union1d(exclude_idx["train"], drop_rows)
```

	split	n_patches	unique	duplicates
0	train	262144	220025	42119
1	valid	32768	28108	4660
2	test	32768	29383	3385

```
train & test : 33
train & valid: 0
valid & test : 0
```

Stratified Sampling of the Training Split

After carrying out several EDA tasks, data preprocessing is performed to prepare the dataset for model training. The first step is to perform sampling on the training split to ensure that the training process remains computationally feasible. To identify the ideal number of samples to retain, the approach by Islam et al. (2024) is referenced, where the study achieved a remarkable result with an accuracy of 95% and an ROC-AUC value of 0.985 by using a working set of approximately 80,000 patches that was split into an 80:20 ratio for training and validation. Therefore, the project has decided to implement a training subset of 70,000 patches to align with existing literature, where 35,000 positive and 35,000 negative samples are drawn from the training split using the code in Figure 12, after excluding patches with a maximum saturation below the 0.07 threshold, as well as those that are duplicated or shared with the testing set. The resulting dataset, together with the full-sized validation and testing splits, is then saved in the `.npy` format to support memory-mapped access, thereby allowing data to be read directly from disk without having to fully load the dataset into memory (Reininghaus, 2023).

Figure 12

Code Snippet to Generate the Stratified Training Subset, Validation Split and Testing Split

```
TRAIN_SUBSET = 70_000                          # Use 70k to maintain computational feasibility
rng = np.random.default_rng(SEED)

y_train = labels["train"]                      # Extract the labels for training, and mark unusable IDs as false
usable = np.ones(len(y_train), dtype=bool)
usable[exclude_idx["train"]] = False

sub_idx = np.sort(np.concatenate([             # Perform sampling by taking half from each positive and negative class
    rng.choice(np.flatnonzero((y_train == label) & usable), TRAIN_SUBSET // 2, replace=False) for label in (0, 1)
]))
sub_y = y_train[sub_idx]

# Loop through all splits
for split in SPLITS:
    tag = f"sub{TRAIN_SUBSET}" if split == "train" else "full"               # Rename the stratified dataset and declare their directory
    x_path = PROCESSED_DIRECTORY / f"{split}_{tag}_x.npy"
    y_path = PROCESSED_DIRECTORY / f"{split}_{tag}_y.npy"
    rows = sub_idx if split == "train" else np.arange(EXPECTED_N[split])     # Create the placeholder image array for the training dataset
    images = np.empty((len(rows), IMG_SIZE, IMG_SIZE, 3), dtype=np.uint8)
    with h5py.File(H5_FILES[f"{split}_x"], "r") as file:                     # Load the actual image data into the array
        dset = file["x"]
        for i in range(0, len(rows), 2048):
            block = rows[i:i + 2048]
            images[i:i + len(block)] = dset[block]
    np.save(x_path, images)                                                  # Save the dataset
    np.save(y_path, labels[split][rows])
    print(f"{split:6} saved {len(rows):,} patches -> {x_path.name}")

# Compile and display the details of the three splits
prepared = {"train (subset)": sub_y, "valid (full)": labels["valid"], "test (full)": labels["test"]}
subset_summary = pd.DataFrame([
    {"split": name, "n_patches": len(y), "negative": int((y == 0).sum()), "positive": int((y == 1).sum()),
     "positive_rate": round(float(y.mean()), 4)}
    for name, y in prepared.items()
])
display(subset_summary)
```

```
train  saved 70,000 patches -> train_sub70000_x.npy
valid  saved 32,768 patches -> valid_full_x.npy
test   saved 32,768 patches -> test_full_x.npy
```

	split	n_patches	negative	positive	positive_rate
0	train (subset)	70000	35000	35000	0.5000
1	valid (full)	32768	16399	16369	0.4995
2	test (full)	32768	16391	16377	0.4998

Computing the Normalisation Statistics

To ensure that gradient-based optimisation techniques can be applied to deep learning models effectively, it is necessary to standardise the pixel values of each patch across the red, green and blue (RGB) channels. Therefore, the mean and standard deviation of each colour channel are calculated using the code in Figure 13 by iterating over the 70,000 patches in the training split, where patches in the validation and testing splits are not included to prevent information in the evaluation dataset from leaking into training. From the results, both red and blue channels have means of 0.7008 and 0.6949 respectively, which are higher than the mean of 0.5393 of the green channel, aligning with the presence of pink and purple colours in the haematoxylin and eosin (H&E) stains to make sentinel lymph nodes visible. The findings also revealed that the normalisation statistics of the PCam dataset are substantially different with statistics from ImageNet, which has a mean of (0.485, 0.456, 0.406) for RGB channels, justifying the decision to not adopt the normalisation approach implemented in similar studies like Mahdi and Nassif (2022) who applied models that are pretrained on ImageNet. Since the project trains its CNN from scratch, dataset-specific statistics will be applied during normalisation to ensure that each channel is centred on 0 with unit variance to enhance optimisation efficiency.

Figure 13

Code Snippet for Computing the Normalisation Statistics of Each Colour Channel

```
# Load the stratified training set
xtr = np.load(PROCESSED_DIRECTORY / f"train_sub{TRAIN_SUBSET}_x.npy", mmap_mode="r")

# Arrays to store sum (total) and sum of squares (total_sq) to calculate std dev
total = np.zeros(3, dtype=np.float64)
total_sq = np.zeros(3, dtype=np.float64)
count = 0

# Loop through images in batches of 2048
for i in range(0, len(xtr), 2048):
    batch = np.asarray(xtr[i:i + 2048], dtype=np.float64) / 255.0   # Rescale the pixel values to the range of [0, 1]
    total += batch.sum(axis=(0, 1, 2))                              # Record the values into sum and sum of squares
    total_sq += np.square(batch).sum(axis=(0, 1, 2))
    count += batch.shape[0] * batch.shape[1] * batch.shape[2]       # Add the number of pixels to count

# Calculate the mean and std dev for each color channel (red, green and blue)
CH_MEAN = total / count
CH_STD = np.sqrt(np.maximum(total_sq / count - CH_MEAN ** 2, 0.0))

# Print the calculations
print("Normalization statistics:")
for channel, name in enumerate(["Red", "Green", "Blue"]):
    print(f"  {name:<6} mean = {CH_MEAN[channel]:.4f},   std = {CH_STD[channel]:.4f}")

# Save the values to a file to be used during training
np.savez(PROCESSED_DIRECTORY / "channel_stats.npz", mean=CH_MEAN.astype(np.float32), std=CH_STD.astype(np.float32))
```

```
Normalization statistics:
  Red    mean = 0.7008,   std = 0.2346
  Green  mean = 0.5393,   std = 0.2763
  Blue   mean = 0.6949,   std = 0.2117
```

Colour Deconvolution and Data Augmentation

To improve model robustness while preventing models from over-relying on staining intensity when performing predictions, data augmentation techniques will be applied during training to introduce variations among patches. To support stain augmentation, colour deconvolution is first applied to separate the original patch into three layers, namely (1) the haematoxylin stain that marks the cell nuclei, (2) the eosin stain that marks protein structures like cytoplasm, and (3) the residual layer that marks the remaining portion, resembling the tissue structure not covered by the stains. With reference to Ruifrok and Johnston (2001), the stain colour matrix and its inverse, which is the colour deconvolution matrix, are defined in Figure 14, allowing sentinel lymph node patches to be deconvolved into three layers, then reconvolved to verify that information loss will not happen throughout the subsequent stain augmentation process.

Figure 14

Code Snippet for Performing Colour Deconvolution on Sentinel Lymph Node Patches

```
STAIN_COLOURS = torch.tensor([                                                  # Matrix to rebuild image from stain
    [0.65, 0.70, 0.29],
    [0.07, 0.99, 0.11],
    [0.27, 0.57, 0.78],
], dtype=torch.float32)
DECONVOLUTION = torch.linalg.inv(STAIN_COLOURS)                                 # Inverse matrix to separate layers

def rgb_to_hed(x):                                                              # Convert patch into layers
    density = -torch.log(x.clamp_min(1e-6))
    return torch.einsum("nchw,cd->ndhw", density, DECONVOLUTION.to(x.device))

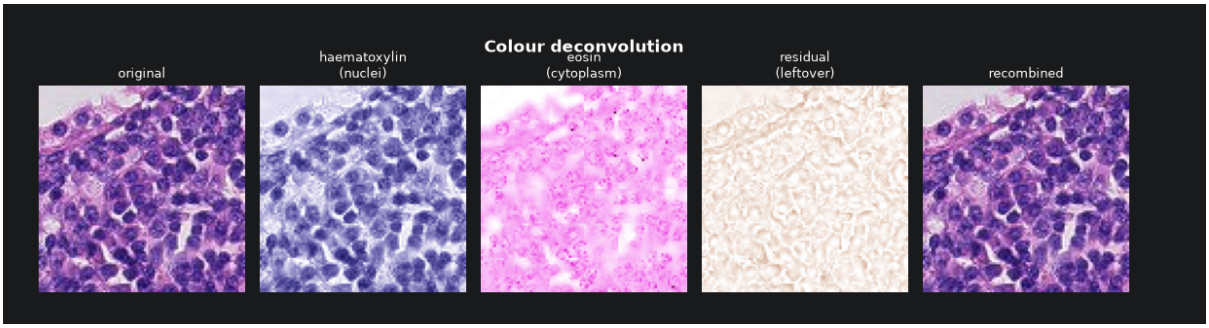
def hed_to_rgb(stains):                                                         # Convert layers back into patches
    density = torch.einsum("nchw,cd->ndhw", stains, STAIN_COLOURS.to(stains.device))
    return torch.exp(-density).clamp(0, 1)

sample = torch.from_numpy(np.ascontiguousarray(xtr[:1])).permute(0, 3, 1, 2).float() / 255   # One patch as sample
stains = rgb_to_hed(sample)

def isolate(index):                                                             # Display deconvoluted images
    only = torch.zeros_like(stains)         # Create an array full of zero
    only[:, index] = stains[:, index]       # Copy the value for a stain to the array
    return hed_to_rgb(only)                 # Convert back into RGB to display

columns = {"original": sample, "haematoxylin\n(nuclei)": isolate(0),            # Dict to store deconvolution details
           "eosin\n(cytoplasm)": isolate(1), "residual\n(leftover)": isolate(2),
           "recombined": hed_to_rgb(stains)}

fig, axes = plt.subplots(1, len(columns), figsize=(2.3 * len(columns), 2.8))    # Display the deconvoluted images
for ax, (name, images) in zip(axes, columns.items()):
    ax.imshow(images[0].permute(1, 2, 0).numpy())
    ax.set_title(name, fontsize=9)
    ax.axis("off")
fig.suptitle("Colour deconvolution", fontweight="bold")
plt.tight_layout()
plt.show()
```



Subsequently, three augmentation operations are implemented to introduce variations among patches, including:

- **D4 dihedral**

  In mathematics, the D4 dihedral group represents the 8 different ways that a square can be flipped and rotated symmetrically (Nallapu & Roychowdhury, 2026). Since patches can be captured at any orientation under real-world scenarios, the `d4_dihedral()` function in Figure 15 applies a transformation randomly selected from the D4 dihedral group to introduce variations in orientation without altering the classification of patches.

  Figure 15

  Code Snippet for Applying Random D4 Dihedral Transformations to Image Patches

```
def d4_dihedral(x):
    for transform in (lambda t: t.transpose(-2, -1),                # Swap rows and columns
                      lambda t: t.flip(-2),                         # Flip upside down
                      lambda t: t.flip(-1)):                        # Flip left to right
        coin = torch.rand(len(x), 1, 1, 1, device=x.device) < 0.5
        x = torch.where(coin, transform(x), x)
    return x
```

- **H&E stain jitter**

  Working upon the layers obtained from colour deconvolution, the `he_stain_jitter()` function in Figure 16 scales and shifts the concentration of the haematoxylin and eosin (H&E) stains by using factors close to 1, thereby reproducing the slight differences in staining that may occur between laboratories.

  Figure 16

  Code Snippet for Applying Haematoxylin and Eosin Stain Jitter

```
def he_stain_jitter(x, strength=0.05):

    # Do nothing if strength is specified as 0
    if strength <= 0:
        return x

    # Deconvolute the patch
    stains = rgb_to_hed(x)

    # Perform scaling and shifting
    shape = (len(x), 2, 1, 1)                                       # Slot to store values
    scale = 1 + (torch.rand(shape, device=x.device) * 2 - 1) * strength     # [ 0, 2] if strength is 1
    shift = (torch.rand(shape, device=x.device) * 2 - 1) * strength         # [-1, 1] if strength is 1

    # Update and return the values for modification
    jittered = torch.cat([stains[:, :2] * scale + shift, stains[:, 2:]], dim=1)
    return hed_to_rgb(jittered)
```

- **Photometric jitter**

  The `photometric_jitter()` function in Figure 17 modifies several image-level properties, namely brightness, contrast, and saturation, allowing the function to replicate different exposure and colour settings that are configured on the whole-slide image (WSI) scanners when laboratories digitise the images of sentinel lymph node tissues.

Figure 17

Code Snippet for Applying Photometric Jitter to Image Patches

```
def photometric_jitter(x, strength=0.15):
    if strength <= 0:                                                    # End for 0 strength
        return x
    factors = 1 + (torch.rand(3, len(x), 1, 1, 1, device=x.device) * 2 - 1) * strength    # Scale factor
    brightness, contrast, saturation = factors
    x = x * brightness                                                   # Update brightness
    mean = x.mean(dim=(1, 2, 3), keepdim=True)                           # Update contrast
    x = (x - mean) * contrast + mean
    gray = rgb_to_grayscale(x)                                           # Update saturation
    x = (x - gray) * saturation + gray
    return x                                                             # Return new value
```

The three augmentation operations are combined into a single function named `augment()` in Figure 18 to demonstrate the combined effect of these operations on the training patches. To show its effects, four patches are retrieved from the training subset and augmented, yielding the results in Figure 19. It is shown that the augmented patches are visibly different from their original image, yet these images still retain the morphological structure of tissues, indicating that data augmentation is able to introduce variation among patches but not distort existing tissue information, hence contributing to model robustness. Therefore, the same three operations, with the same order, will be applied during model training to introduce variation among patches to mitigate the limitation that the PCam dataset is sourced from only two centres.

Figure 18

Code Snippet for Combining the Three Augmentation Operations

```
def augment(patches, d4=True, hed_strength=0.05, photometric=0.15):

    x = patches.float() / 255.0 if patches.dtype == torch.uint8 else patches    # Perform rescaling on pixel values

    if d4:                                                               # Apply augmentation techniques
        x = d4_dihedral(x)
    x = he_stain_jitter(x, strength=hed_strength)
    x = photometric_jitter(x, strength=photometric)

    return x.clamp(0, 1)                                                 # Return the augmented value
```

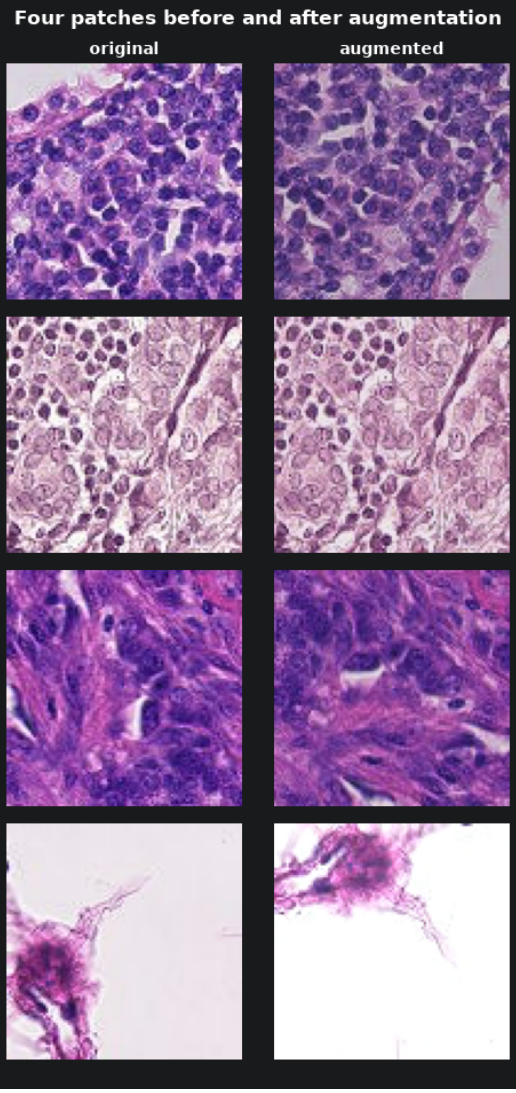
Figure 19

Code Snippet for Demonstrating the Augmentation of Training Patches

```
# Select 4 patches
patches = torch.from_numpy(np.ascontiguousarray(xtr[:4])).permute(0, 3, 1, 2)

# Apply augmentation
torch.manual_seed(0)
columns = {
    "original": augment(patches, d4=False, hed_strength=0, photometric=0),
    "augmented": augment(patches),
}

# Show output of the 4 patches
fig, axes = plt.subplots(4, 2, figsize=(5, 9.5))
for col, (name, images) in enumerate(columns.items()):
    images = images.permute(0, 2, 3, 1).numpy()
    for row in range(4):
        axes[row, col].imshow(images[row])
        axes[row, col].set_xticks([]); axes[row, col].set_yticks([])
        for spine in axes[row, col].spines.values():
            spine.set_visible(False)
    axes[0, col].set_title(name, fontsize=10, fontweight="bold")
fig.suptitle("Four patches before and after augmentation", fontweight="bold")
plt.tight_layout()
plt.show()
```



Final Verification of the Prepared Dataset

Before moving to the data modelling phase, the prepared dataset is verified one final time using the code snippet in Figure 20 to ensure that it does not contain any empty, invalid, duplicated, leaked, or low-information data. Based on the results in Figure 21, it is shown that the training split is fully cleaned with a perfect class distribution, while duplicates and low-saturation patches in the validation and testing splits are retained, not only to align with real-world scenarios where input data may not be perfectly clean for predictions, but also to ensure that the reported metrics are comparable with existing works on the PCam dataset to demonstrate the reliability of predictive results from the deep learning models.

Figure 20

Code Snippet for Performing the Final Verification of the Prepared Dataset

```
# Retrieve the saved .npy files
prepared = {}
for split in SPLITS:
    tag = f"sub{TRAIN_SUBSET}" if split == "train" else "full"
    prepared[split] = (
        np.load(PROCESSED_DIRECTORY / f"{split}_{tag}_x.npy", mmap_mode="r"),
        np.load(PROCESSED_DIRECTORY / f"{split}_{tag}_y.npy"),
    )

# Perform hashing to evaluate for duplicates and leakage
prepared_hashes = {}
for split, (images, _) in prepared.items():
    h = []
    for i in range(0, len(images), 2048):
        for image in images[i:i + 2048]:
            h.append(hashlib.sha256(image.tobytes()).hexdigest())
    prepared_hashes[split] = pd.Series(h)

# Calculate the maximum saturation of every patch
prepared_saturation = {}
for split, (images, _) in prepared.items():
    values = []
    for i in range(0, len(images), 2048):
        x = images[i:i + 2048].astype(np.float32) / 255.0
        mx, mn = x.max(axis=3), x.min(axis=3)
        sat = (mx - mn) / np.maximum(mx, 1e-6)
        values.append(sat.reshape(len(x), -1).max(axis=1))
    prepared_saturation[split] = np.concatenate(values)

# Perform checking on each split
report = []
for split, (images, labels_split) in prepared.items():
    h = prepared_hashes[split]
    other = set().union(*(set(prepared_hashes[s]) for s in SPLITS if s != split))
    report.append({
        "split": split,
        "patches": len(images),
        "shape": str(images.shape[1:]),
        "dtype": str(images.dtype),
        "blank_patches": int((images.reshape(len(images), -1).max(axis=1) == 0).sum()),
        "invalid_labels": int((~np.isin(labels_split, [0, 1])).sum()),
        "duplicates": int(h.duplicated().sum()),
        "shared_with_other_splits": int(h.isin(other).sum()),
        "below_saturation_0.07": int((prepared_saturation[split] < SATURATION_THRESHOLD).sum()),
        "negative": int((labels_split == 0).sum()),
        "positive": int((labels_split == 1).sum()),
        "positive_rate": round(float(labels_split.mean()), 4),
    })

# Display the results
checks = pd.DataFrame(report).set_index("split")
display(checks.T)

# Also display the normalization statistics
stats = np.load(PROCESSED_DIRECTORY / "channel_stats.npz")
print(f"Channel means : {stats['mean'].round(4)}")
print(f"Channel stds  : {stats['std'].round(4)}")
```

Figure 21

Verification Results for the Prepared Training, Validation and Testing Splits

split	train	valid	test
patches	70000	32768	32768
shape	(96, 96, 3)	(96, 96, 3)	(96, 96, 3)
dtype	uint8	uint8	uint8
blank_patches	0	0	0
invalid_labels	0	0	0
duplicates	0	4660	3385
shared_with_other_splits	0	0	0
below_saturation_0.07	0	55	62
negative	35000	16399	16391
positive	35000	16369	16377
positive_rate	0.5	0.4995	0.4998

```
Channel means : [0.7008 0.5393 0.6949]
Channel stds  : [0.2346 0.2763 0.2117]
```

Model Building

Environment and Preprocessing Setup

Since model training is performed in a separate Jupyter notebook, the first step of the model building phase is to reconfigure the environment. Several configurations, such as the library imports and dataset directories involved in model training, are shown in Figure 22, while Figure 23 displays preprocessing functions like normalisation and augmentation that will be applied to image patches during the model training process.

Figure 22

Code Snippet for Configuring the Environment and Loading the Prepared Dataset

```
# Library imports
import random
import time
import warnings
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from sklearn.metrics import (
    accuracy_score,
    average_precision_score,
    brier_score_loss,
    confusion_matrix,
    f1_score,
    matthews_corrcoef,
    roc_auc_score,
    roc_curve,
)
from sklearn.calibration import calibration_curve

# Seed
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

# Device
DEVICE = torch.device(
    "mps" if torch.backends.mps.is_available()
    else "cuda" if torch.cuda.is_available()
    else "cpu"
)

# Warnings filter
warnings.filterwarnings("ignore", category=UserWarning)

# Directory
PROCESSED = Path("..") / "data" / "processed"
xtr       = np.load(PROCESSED / "train_sub70000_x.npy", mmap_mode="r")
ytr       = np.load(PROCESSED / "train_sub70000_y.npy").astype(np.float32).reshape(-1)
xva       = np.load(PROCESSED / "valid_full_x.npy", mmap_mode="r")
yva       = np.load(PROCESSED / "valid_full_y.npy").astype(np.float32).reshape(-1)
stats     = np.load(PROCESSED / "channel_stats.npz")
mean_t    = torch.tensor(stats["mean"], dtype=torch.float32, device=DEVICE)
std_t     = torch.tensor(stats["std"], dtype=torch.float32, device=DEVICE)
```

Figure 23

Code Snippet for Defining the Preprocessing Functions Applied During Model Training

```python
# Stain color matrix from Ruifrok and Johnston (2001)
STAIN_COLOURS = torch.tensor([
    [0.65, 0.70, 0.29],
    [0.07, 0.99, 0.11],
    [0.27, 0.57, 0.78],
], dtype=torch.float32)

# Color-deconvolution matrix to split patches into layers
DECONVOLUTION = torch.linalg.inv(STAIN_COLOURS)

# Convert pixel values to the range of [0, 1]
def to_float(x):
    return x.float() / 255.0 if x.dtype == torch.uint8 else x

# Separate image to layers
def rgb_to_hed(x):
    density = -torch.log(x.clamp_min(1e-6))
    return torch.einsum("nchw,cd->ndhw", density, DECONVOLUTION.to(x.device))

# Combine layers into image
def hed_to_rgb(stains):
    density = torch.einsum("nchw,cd->ndhw", stains, STAIN_COLOURS.to(stains.device))
    return torch.exp(-density).clamp(0, 1)

# Flipping and rotation
def d4_dihedral(x):
    for transform in (lambda t: t.transpose(-2, -1),
                      lambda t: t.flip(-2),
                      lambda t: t.flip(-1)):
        coin = torch.rand(len(x), 1, 1, 1, device=x.device) < 0.5
        x = torch.where(coin, transform(x), x)
    return x

# Modify the intensity of stains
def he_stain_jitter(x, strength=0.05):
    if strength <= 0:
        return x
    stains = rgb_to_hed(x)
    shape = (len(x), 2, 1, 1)
    scale = 1 + (torch.rand(shape, device=x.device) * 2 - 1) * strength
    shift = (torch.rand(shape, device=x.device) * 2 - 1) * strength
    return hed_to_rgb(torch.cat([stains[:, :2] * scale + shift, stains[:, 2:]], dim=1))

# Alter picture-related settings
def photometric_jitter(x, strength=0.15):
    if strength <= 0:
        return x
    factors = 1 + (torch.rand(3, len(x), 1, 1, 1, device=x.device) * 2 - 1) * strength
    brightness, contrast, saturation = factors
    x = x * brightness
    mean = x.mean(dim=(1, 2, 3), keepdim=True)
    x = (x - mean) * contrast + mean
    gray = (0.299 * x[:, 0] + 0.587 * x[:, 1] + 0.114 * x[:, 2]).unsqueeze(1)
    return (x - gray) * saturation + gray

# Crop patches to the central region (32 x 32 for PCam) for normalization
def centre_crop(x, size):
    h = x.shape[-2]
    if size >= h:
        return x
    off = (h - size) // 2
    return x[..., off:off + size, off:off + size]

# z-score normalization
def normalise(x, mean, std):
    return ((x - mean.view(1, 3, 1, 1)) / std.view(1, 3, 1, 1)).contiguous()
```

Model Architecture

This project designs two neural network architectures to support the binary classification of sentinel lymph node patches, where convolutional architectures are chosen to allow these models to automatically pick up the different morphological and structural characteristics of the lymph node patches.

Vanilla Convolutional Neural Network (CNN)

A vanilla convolutional neural network (CNN) is defined in Figure 24, acting as a baseline to compare against the performance of the hybrid CNN-BiLSTM model on the PCam dataset.

Figure 24

Code Snippet for Defining the Architecture of the Vanilla CNN

```
class BaselineCNN(nn.Module):

    # Declares the components inside the model: head, features and classifier
    def __init__(self,
                 widths=(32, 64, 128, 256),     # Number of 3x3 filters
                 dropout=0.3,                   # Random zeroing on dense head during training
                 block_dropout=0.05,            # Random zeroing on feature map during training
                 head="gap",                    # Determines where the decision is read from (gap, center, both)
                 fc_dim=128):                   # Size of dense layer (hidden layers)

        # Initialize the neural network module in PyTorch
        super().__init__()

        # Construct the head used by NN later
        self.head = head

        # The layers involved in the baseline CNN, with 3 channels in the beginning
        layers, prev = [], 3

        # Loop through each filter block
        for w in widths:
            layers += [nn.Conv2d(prev, w, 3, padding=1, bias=False),    # prev = channels, w = number of filters & output count
                       nn.BatchNorm2d(w),                               # Perform normalization on the outputs
                       nn.ReLU(inplace=True),                           # Use ReLU to remove negative values
                       nn.Conv2d(w, w, 3, padding=1, bias=False),       # Perform convolution again to learn from adjacent
                       nn.BatchNorm2d(w),                               # Perform normalization again
                       nn.ReLU(inplace=True),                           # Remove negative values again
                       nn.MaxPool2d(2),                                 # Use a 2x2 filter to retain the max value
                       nn.Dropout2d(block_dropout)]                     # Ignore 5% of outputs to avoid overreliance
            prev = w                                                    # Once done, update the current channel number

        # Convert the layers into steps (these are the hidden layers)
        self.features = nn.Sequential(*layers)

        # Classifier to determine the prediction
        self.classifier = nn.Sequential(
            nn.Linear(prev * (2 if head == "both" else 1), fc_dim),     # Combine the channels into fc_dim
            nn.BatchNorm1d(fc_dim),                                     # Normalize output
            nn.ReLU(inplace=True),                                      # Remove negative values
            nn.Dropout(dropout),                                        # Ignore 30% of features
            nn.Linear(fc_dim, 1),                                       # Convert the final hidden layer into a result
        )

    # Defines the flow of the model
    def forward(self, x):
        f = self.features(x)                                            # Run inputs over the layers
        gap = f.mean(dim=(-2, -1))                                      # Gap: get average by combining each value in each channel
        centre = f[..., 2:4, 2:4].mean(dim=(-2, -1))                    # Center: get the average by combining the center region

        # Determine which one goes into the classifier
        z = gap if self.head == "gap" else centre if self.head == "centre" else \
            torch.cat([gap, centre], dim=1)

        return self.classifier(z).squeeze(-1)                           # Return the final prediction
```

The baseline CNN implemented in this project is designed based on the work of Simonyan and Zisserman (2015), where their VGG model implements two convolutional layers within a filter block with channel counts doubled after each pooling operation. Despite its simple design, their CNN model is able to outperform contemporary architectures like GoogLeNet by Szegedy et al. (2014, as cited in Simonyan & Zisserman, 2015), thereby encouraging the adoption of the CNN model into the current project. The baseline CNN implements two components, namely:

- **Feature extractor**

  The baseline CNN model implements four convolutional blocks with filter numbers of 32, 64, 128 and 256, allowing it to capture the increasingly complex structures of lymph node patches while gradually reducing the spatial resolution. Each block applies two consecutive convolutional layers that use their own set of 3×3 filters, so that each position in the output is able to gain a higher coverage on its neighbouring elements without having to introduce a large number of parameters while introducing non-linearity to represent complex structures more accurately. This is followed by batch normalisation to rescale the activations and improve the stability of the training process (Bjorck et al., 2018), with a ReLU activation function applied to enhance the capability of deep learning models to capture non-linear relationships between the stacked layers. At the end of each filter block, the spatial dimension of the image is reduced by applying the maximum pooling operation that uses a 2×2 filter to aggregate elements so that the computational cost of subsequent operations can be reduced. A dropout mechanism is also applied to randomly remove feature maps during the training process so that models will not become too reliant on individual features and improve their generalisability (Quirin et al., 2021).

- **Classifier head**

  Once the feature extractor component produces the final feature map, the classifier head aggregates it into a 256-dimensional vector, either by global average pooling across all positions (`gap`), or by averaging the central 2×2 positions to match PCam's 32×32 label region (`centre`), with the baseline adopting the latter. Unlike the conventional flattening approach that introduces millions of parameters in the dense layer (Jeczmionek & Kowalski, 2021), this helps to reduce overfitting risks. The vector is then passed into the classifier to reduce the feature sets into 128 units, before batch normalisation, the ReLU activation function, and the dropout mechanism are applied to improve model generalisability. The units are then combined into a final dense layer that produces a single logit value that will be converted into probabilities to support model prediction.

CNN-Bidirectional Long Short-Term Memory (CNN-BiLSTM)

As an extension to the baseline CNN model, a hybrid CNN-BiLSTM is also implemented in this project, with the code snippet in Figure 25 showing its architecture.

Figure 25

Code Snippet for Defining the Architecture of the Hybrid CNN-BiLSTM

```
# The class that handles the additive attention mechanism
class AttentionPool(nn.Module):

    def __init__(self, dim, hidden=128):                        # Declare the components inside the attention mechanism
        super().__init__()
        self.score = nn.Sequential(nn.Linear(dim, hidden),      # Attention score. Combine into 128 features
                                   nn.Tanh(),                   # Use tanh: range [-1, 1]
                                   nn.Linear(hidden, 1))        # Combine into one output

    def forward(self, h):                                # Define how does the attention mechanism work
        w = torch.softmax(self.score(h), dim=1)          # Calculate the attention score and convert them to fractions that add to 1
        return (w * h).sum(dim=1), w.squeeze(-1)         # Return the weighted average and the original weights for each element

# The class that handles the CNN-BiLSTM model
class CNNBiLSTM(nn.Module):

    def __init__(self,                                   # Define the components inside the model
                 widths=(32, 64, 128, 256),              # Number of filters for each block
                 grid=6,                                 # Size of map before feeding into LSTM
                 hidden=256,                             # Size of LSTM memory
                 num_layers=1,                           # Number of LSTM layers
                 dropout=0.3,                            # Percentage of features to ignore in LSTM (if stacked) and classifier
                 block_dropout=0.05,                     # Percentage of features to ignore in conv blocks
                 fc_dim=128,                             # Size of the dense layer (hidden layers)
                 head="attn"):                           # How the final decision is made (attn, center, both)
        super().__init__()                                                  # Constructors and declaration of local variables
        self.head = head
        self.grid = grid
        layers, prev = [], 3                                                # Create layers based on the filter block
        for w in widths:
            layers += [nn.Conv2d(prev, w, 3, padding=1, bias=False),        # prev = channel, w = filter count & output channel count
                       nn.BatchNorm2d(w),                                   # Normalization on result
                       nn.ReLU(inplace=True),                               # Remove negative values
                       nn.Conv2d(w, w, 3, padding=1, bias=False),           # Additional conv layer to learn more about image
                       nn.BatchNorm2d(w),                                   # Normalization
                       nn.ReLU(inplace=True),                               # Remove negative values
                       nn.MaxPool2d(2),                                     # Max pooling on 2x2 window
                       nn.Dropout2d(block_dropout)]                         # Ignore 5% of the features
            prev = w
        self.features = nn.Sequential(*layers)                                  # Add the layers to features
        self.pool = nn.AdaptiveAvgPool2d((grid, grid))                          # A pool to reduce the layer size to cater for LSTM
        self.rnn = nn.LSTM(input_size=prev,                                     # The LSTM component. Input size
                           hidden_size=hidden,                                  # The size of features in LSTM layers
                           num_layers=num_layers,                               # Number of LSTM layers
                           batch_first=True,                                    # Data shape becomes (batch, step/pixel, feature)
                           bidirectional=True,                                  # Two-way processing
                           dropout=dropout if num_layers > 1 else 0.0,          # Only apply dropout if more than one LSTM layer
        )
        self.attn = (AttentionPool(2 * hidden, hidden=fc_dim)                   # The attention component
                     if head in ("attn", "both") else None)
        self.classifier = nn.Sequential(                                        # The classifier
            nn.Linear(2 * hidden * (2 if head == "both" else 1), fc_dim),       # Convert layer output to fc_dim
            nn.BatchNorm1d(fc_dim),                                             # Normalization
            nn.ReLU(inplace=True),                                              # ReLU to remove negative values
            nn.Dropout(dropout),                                                # Ignore 30% of the features
            nn.Linear(fc_dim, 1),                                               # Output a logit to be converted later
        )

    def forward(self, x, return_attention=False):               # Define the flow of the model
        f = self.pool(self.features(x))                         # Run the convolutional layers and pool output to the intended size
        b, c, g, _ = f.shape                                    # Sequence construction: Rearrange the outputs for LSTM
        seq = f.permute(0, 2, 3, 1).reshape(b, g * g, c)
        h, _ = self.rnn(seq)                                    # Run the LSTM component
        attn = None                                             # Run the attention component
        if self.head in ("attn", "both"):
            ctx, attn = self.attn(h)
            attn = attn.view(b, g, g)
        if self.head in ("centre", "both"):                     # If center region involved, reshape weights and take middle region
            m = g // 2
            centre = h.view(b, g, g, -1)[:, m - 1:m + 1, m - 1:m + 1].mean(dim=(1, 2))
        z = (ctx if self.head == "attn" else                    # Determine the output to be passed to the classifier
             centre if self.head == "centre" else
             torch.cat([ctx, centre], dim=1))
        logits = self.classifier(z).squeeze(-1)                 # Run the classifier
        return (logits, attn) if return_attention else logits   # Return the results
```

The baseline CNN model averages the final feature map over the centre region of each patch without considering its spatial arrangement, making the baseline CNN model unable to capture the relationships across the different regions of a patch. Since Srikantamurthy et al. (2023) highlight that hybrid CNN-LSTM models are able to overcome this limitation and achieve better performance on breast histopathology datasets when compared with conventional baseline CNN models, this project implements a hybrid CNN-BiLSTM model to evaluate the impact of the recurrent component in improving the performance of deep learning models on the PCam dataset. The hybrid model consists of several components, including:

- **Feature extractor**

  The convolutional trunk applied in the CNN-BiLSTM model is kept identical to the baseline CNN model. Similar to the vanilla CNN, the CNN-BiLSTM also implements four convolution blocks with filter counts of 32, 64, 128 and 256, together with the same paired convolutional layers, batch normalisation, ReLU activation, maximum pooling, and dropout mechanism applied within each filter block. This is to ensure that the recurrent aggregation strategy applied in the CNN-BiLSTM model is the only component that differs from the baseline CNN model, so that the difference in model performance can be fairly attributed to the aggregation strategy while ensuring that the quality of the extracted features in both models remains consistent. Therefore, the convolutional trunk also outputs a $6 \times 6 \times 256$ feature map that is identical to the baseline CNN model before subsequent aggregation operations are applied.

- **Sequence construction**

  In the sequence construction component, the feature map produced by the convolution trunk is flattened in raster order to preserve the spatial arrangement of the regions in the image patches. All the 36 positions in the feature map are read from left to right and from top to bottom, then transformed into a sequence of 36 steps, where each step carries the 256 features that describe the characteristics of a region in the patch. Unlike the read-out operation of the baseline CNN that collapses its selected positions into a single average, the flattening technique implemented in the sequence construction phase allows the ordering of each position to be retained, thereby allowing the subsequent recurrent component to process each region in a sequential manner and potentially contributing to the performance of the deep learning models when handling medical images (Carvalho et al., 2022).

- **Bidirectional LSTM**

  The 36-step sequence is passed into a bidirectional long short-term memory (BiLSTM) layer that is configured with a hidden size of 256 for each direction. When iterating through each sequence step, the LSTM implements a gating mechanism to update its internal memory state, allowing each step to possess information that is obtained during an earlier step (Malashin et al., 2024). Since the layer is bidirectional, the LSTM traverses the sequence in both forward and backward directions, further providing additional context to each region regarding the other regions that precede and follow it. The outputs of both directions are then concatenated, yielding an output of 512 features for each step.

- **Read-out (attention pooling or centre)**

  The hybrid model provides two read-out options to summarise the 36-step sequence into a single vector. The first is an additive attention mechanism that references the approach of Ilse et al. (2018), where each step is passed into a small scoring network that assigns it a single score, then the scores are normalised using a Softmax function so that they sum to one. Each step is then scaled by its normalised score before the results are summed across the sequence to calculate the weighted average for each of the 512 features, thereby allowing the regions containing stronger evidence of metastatic tissue to contribute more heavily to the resultant vector (Tavolara et al., 2022) compared to a uniform average. The second option is to average only the BiLSTM outputs at the four central positions, which is applied in the baseline CNN-BiLSTM run to match PCam's label region. Nonetheless, the attention pooling mechanism is still evaluated as a candidate during model tuning.

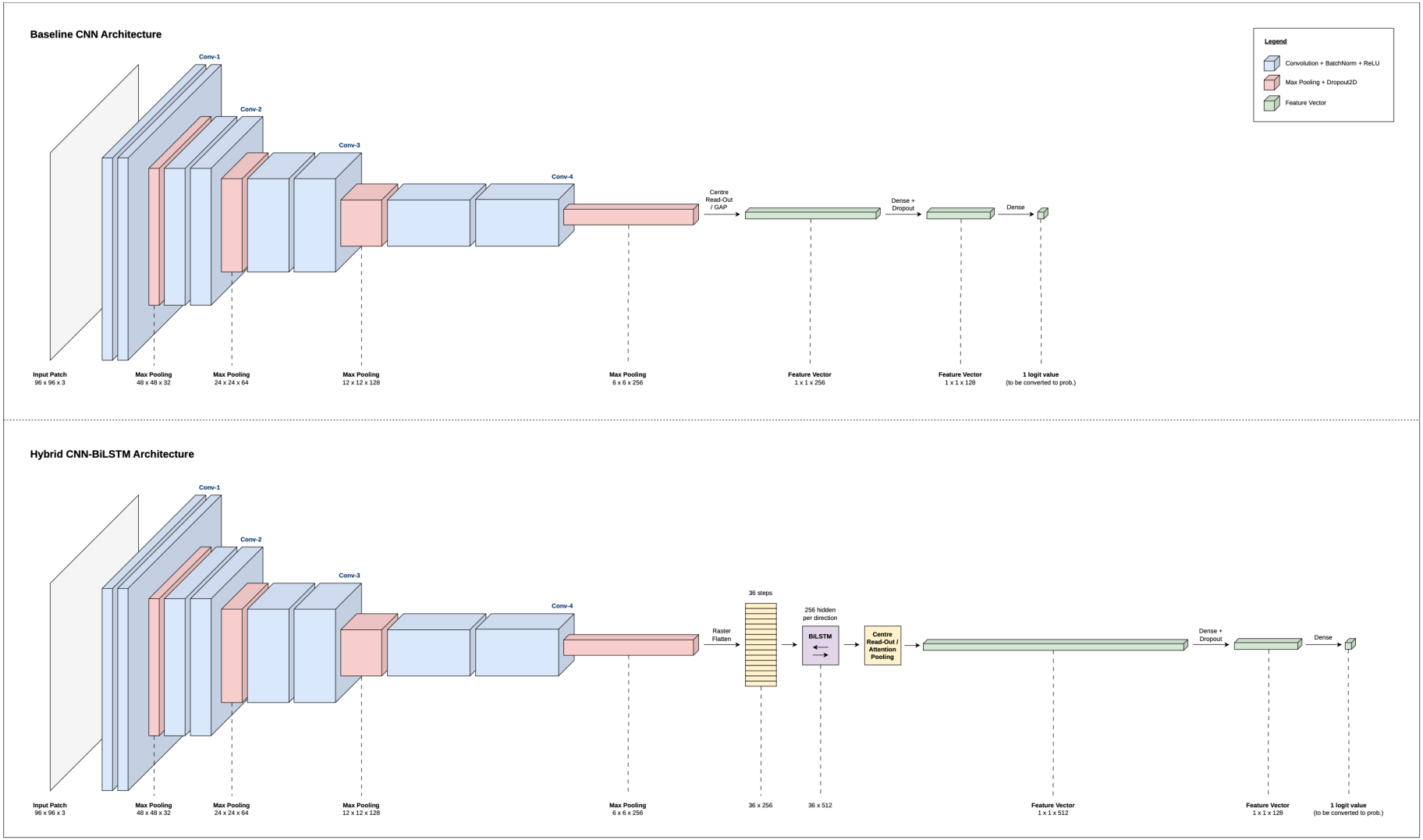
- **Classifier head**

  Lastly, the 512-dimensional summary vector from the read-out is passed to a classifier head that has the same structure as the baseline CNN model, where the 512-dimensional vector is first reduced into a dense layer with 128 units, followed by operations like batch normalisation, a ReLU activation and a dropout mechanism, before a final dense layer is implemented to map all the 128 units to a single logit value that will be converted into a probability to support the binary classification of sentinel lymph node patches.

The architectural diagrams for both the baseline CNN and the hybrid CNN-BiLSTM models are shown in Figure 26, providing a clearer overview of the components that are involved in both models implemented in this project.

Figure 26

Architectural Diagrams of the Baseline CNN and the Hybrid CNN-BiLSTM



Model Training

Using the designed architectures, the two neural network models are trained on the stratified subset of PCam's training split with 70,000 image patches.

Training Configuration

Several functions are defined to facilitate the model training process, where the `predict()` function in Figure 27 applies a trained model to a set of image patches, then converts the logit value produced for each patch into probabilities. On the other hand, the `metrics()` function in Figure 28 uses the calculated probabilities to compute evaluation metrics, such as accuracy, ROC-AUC, and F1 score, for the corresponding model.

Figure 27

Code Snippet for Generating Predicted Probabilities Using a Trained Model

```
@torch.no_grad()
def predict(model, images, labels, *, batch_size=512, input_size=96):
    model = model.to(DEVICE).eval()                                          # Evaluation mode (no dropout)
    out_probs = []
    for start in range(0, len(labels), batch_size):                          # Loop through each batch
        xb = torch.from_numpy(np.ascontiguousarray(images[start:start + batch_size]))  # Retrieve image data
        xb = to_float(xb.permute(0, 3, 1, 2).contiguous().to(DEVICE))        # Perform reshape
        xv = normalise(centre_crop(xb, input_size), mean_t, std_t)           # Perform normalization
        out = model(xv)                                                      # Run the model for prediction
        out = out[0] if isinstance(out, tuple) else out                      # Only take the logits
        out_probs.append(torch.sigmoid(out.float()).cpu().numpy())           # Sigmoid (convert to prob.)
    return np.concatenate(out_probs), np.asarray(labels, dtype=np.float32)   # Record and return outputs
```

Figure 28

Code Snippet for Computing the Evaluation Metrics from Predicted Probabilities

```
def metrics(probs, targets, threshold=0.5):
    y = np.asarray(targets).astype(int)                                      # Retrieve labels
    p = np.clip(np.asarray(probs, dtype=np.float64), 1e-7, 1 - 1e-7)         # Cap prob to avoid errors
    pred = (p >= threshold).astype(int)                                      # Convert prob into predictions
    tn, fp, fn, tp = confusion_matrix(y, pred, labels=[0, 1]).ravel()        # Calculate confusion matrix
    sens, spec = tp / max(tp + fn, 1), tn / max(tn + fp, 1)                  # Calculate TPR and TNR

    # Return all metrics here
    return {
        "accuracy": float(accuracy_score(y, pred)),
        "balanced_accuracy": float((sens + spec) / 2),
        "auc": float(roc_auc_score(y, p)),
        "average_precision": float(average_precision_score(y, p)),
        "f1": float(f1_score(y, pred)),
        "mcc": float(matthews_corrcoef(y, pred)),
        "sensitivity": float(sens), "specificity": float(spec),
        "precision": float(tp / max(tp + fp, 1)),
        "nll": float(-(y * np.log(p) + (1 - y) * np.log(1 - p)).mean()),
        "brier": float(brier_score_loss(y, p)),
        "threshold": float(threshold),
        "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp), "n": int(len(y)),
    }
```

Figure 29

Code Snippet for Training the Neural Networks

```
def train(model, name, train_data, val_data, *, epochs=30, batch_size=256, lr=3e-4,
          weight_decay=1e-4, label_smoothing=0.05, aug_hed_sigma=0.05,
          aug_photometric=0.15, input_size=96, seed=42, verbose=True):

    random.seed(seed)                                                    # Define seed for reproducibility
    np.random.seed(seed)
    torch.manual_seed(seed)

    model = model.to(DEVICE)                                             # Device (GPU/CPU)

    x_train, y_train = train_data                                        # Get training and validation
    x_val, y_val = val_data

    steps_per_epoch = len(y_train) // batch_size                         # Determine the number of batches

    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)       # Optimizer: AdamW

    criterion = nn.BCEWithLogitsLoss()                                   # Loss function: logit

    # Details to record throughout training
    history = {k: [] for k in ("epoch", "train_loss", "train_acc", "val_loss",
                               "val_acc", "val_auc", "seconds")}
    best = {"epoch": -1, "val_auc": -np.inf, "val_acc": 0.0, "val_nll": np.inf}
    best_state = None
    rng = np.random.default_rng(seed)
    for epoch in range(1, epochs + 1):
        t0 = time.time()                                                 # Begin model training
        model.train()
        run_loss, run_correct, run_n = 0.0, 0, 0
        order = rng.permutation(len(y_train))                            # Randomize arrangement of patch
        for start in range(0, steps_per_epoch * batch_size, batch_size):

            # Retrieve the images
            sel = np.sort(order[start:start + batch_size])               # Sorting makes processing faster
            xb = torch.from_numpy(np.ascontiguousarray(x_train[sel]))    # Load image data
            xb = xb.permute(0, 3, 1, 2).contiguous().to(DEVICE)          # Reshape to correct shape
            yb = torch.from_numpy(y_train[sel]).to(DEVICE)               # Extract the labels

            # Preprocessing on image
            x = d4_dihedral(to_float(xb))                                # Flips and rotations
            x = he_stain_jitter(x, strength=aug_hed_sigma)               # Stain-colour jitter
            x = photometric_jitter(x, strength=aug_photometric)          # Brightness / contrast
            x = normalise(centre_crop(x.clamp(0, 1), input_size), mean_t, std_t)     # Normalization
            target = yb * (1 - label_smoothing) + 0.5 * label_smoothing  # Label smoothing

            # Optimizer (controls learning rate)
            optimizer.zero_grad(set_to_none=True)                        # Clear old gradients
            out = model(x)                                               # Forward pass
            loss = criterion(out, target)                                # Compute loss
            loss.backward()                                              # Backpropagate
            optimizer.step()                                             # Update weights

            # Record loss (for reporting)
            run_loss += loss.item() * len(sel)                           # Calculate total loss
            run_correct += ((out.detach() >= 0).float() == yb).sum().item()          # Count no. of correct predictions
            run_n += len(sel)                                            # Record the number of images seen

        # Training done. Proceed with validation
        val_probs, _ = predict(model, x_val, y_val, input_size=input_size)           # Fit model into validation set
        m = metrics(val_probs, y_val)                                    # Retrieve the metrics

        for key, value in (("epoch", epoch), ("train_loss", run_loss / run_n),       # Record the epoch detail
                           ("train_acc", run_correct / run_n), ("val_loss", m["nll"]),
                           ("val_acc", m["accuracy"]), ("val_auc", m["auc"]),
                           ("seconds", time.time() - t0)):
            history[key].append(value)

        if m["auc"] > best["val_auc"]:                                   # Record if AUC improved
            best = {"epoch": epoch, "val_auc": m["auc"], "val_acc": m["accuracy"],
                    "val_nll": m["nll"]}
            best_state = {k: v.detach().cpu().clone() for k, v in model.state_dict().items()}

        if verbose:                                                      # Print details if verbose is on
            print(f"    ep {epoch:>3}/{epochs}  loss {run_loss / run_n:.4f} "
                  f"acc {run_correct / run_n:.4f} | val acc {m['accuracy']:.4f} "
                  f"auc {m['auc']:.4f}  {time.time() - t0:.0f}s", flush=True)

    model.load_state_dict(best_state)                                    # Restore the best epoch

    if verbose:                                                          # Print details if verbose is on
        print(f"  best epoch {best['epoch']}: val AUC {best['val_auc']:.4f}")

    params = dict(epochs=epochs, batch_size=batch_size, lr=lr, weight_decay=weight_decay,      # Store the best parameters
                  label_smoothing=label_smoothing, aug_hed_sigma=aug_hed_sigma,
                  input_size=input_size, seed=seed)
    return {"history": history, "best": best, "params": params}
```

The `train()` function, outlining the steps involved in training the neural network models, is defined in Figure 29. Each component of the training process is broken down as follows:

- **Define the optimiser and loss function**

  After setting up fundamental configurations, such as the random seed and the number of batches per epoch, AdamW is configured as the optimiser due to its efficient optimisation and regularisation capabilities (Zhou et al., 2024). The optimiser is then configured with a learning rate of 3×10^{-4} to allow weights to be updated in small steps to mitigate the issue of overshooting, together with a weight decay of 10^{-4} that provides regularisation by imposing a small penalty on large values to reduce the risk of overfitting. For the loss function, binary cross-entropy with logits is selected as it is able to measure the difference between the predicted and actual binary outcomes (Terven et al., 2025), making it suitable for binary classification tasks.

- **Loop through each epoch**

  Both the CNN and CNN-BiLSTM models are trained for 30 epochs, where each epoch involves one complete iteration through the training set. This is to ensure that neural network models are able to progressively capture image patterns from the training patches and consistently update their parameters to improve performance (Mahmoud et al., 2024). However, to prevent models from relying on a fixed order of samples during training, the patches are randomly shuffled, thereby improving the trained models' performance and learning capabilities (Mao et al., 2022).

- **Loop through each batch**

  Within each epoch, to improve the computational efficiency of the model training process, the training patches are processed in batches of 256 to allow multiple samples to be processed in parallel. Each batch will first undergo several preprocessing steps, including data augmentation like H&E stain jitter and photometric jitter to introduce variation among the training images, as well as normalisation to support a more stable optimisation process. Additionally, label smoothing is applied to the labels so that they are adjusted away from 0 and 1 to improve model calibration and reduce overconfidence (Liu et al., 2022). The models will then be trained using these processed patches, where the logit values for each patch are computed together with the calculation of associated loss and the backpropagation of gradients to iteratively update the weights of the neural network.

- **Validate the model**

  Once each epoch is completed, the validation split is used to evaluate the performance of the model at the current epoch, with the resulting evaluation metrics recorded in the training history.

- **Restore the best-performing epoch**

  Based on the results obtained from the validation split, if the current epoch outperforms the best performance recorded from the previous epochs on the validation ROC-AUC metric, the epoch number and its validation metrics will be recorded in the `best` dictionary. The validation ROC-AUC is used as the selection criterion as it is able to reflect the ability of models to distinguish between positive and negative classes (Cabot & Ross, 2023), aligning with the aim of the current project to classify the presence of metastatic tissues in sentinel lymph node patches. With the model state of the best-performing epoch stored, it can be reloaded after training so that the final model is able to represent the best validation performance of the model throughout the training process.

- **Return the training record**

  Lastly, the `train()` function returns several pieces of information, such as the training history of each epoch, the validation metrics of the best-performing epoch, as well as the corresponding training parameters. These results can be used to perform further analysis, including plotting the learning curves and comparing the performance of different models, thereby facilitating the subsequent model evaluation phase.

Training Baseline CNN and Hybrid CNN-BiLSTM

With all the necessary configurations established, both the baseline CNN and hybrid CNN-BiLSTM models are trained using the `train()` function, as shown in Figures 30 and 31. Both models are configured with `head="centre"`, indicating that the final prediction will be based on the features at the centre of each image patch, thereby aligning with the labelling rule of the PCam dataset. On average, each epoch takes approximately one minute, causing both models to consume around 30 to 35 minutes to complete their training across 30 epochs.

Figure 30

Code Snippet for Creating and Training the Baseline CNN Model

```
torch.manual_seed(42)                                                    # Fix a seed
model_baseline = BaselineCNN(dropout=0.3, block_dropout=0.05, head="centre")   # Create the baseline model
run_baseline = train(model_baseline, "cnn", (xtr, ytr), (xva, yva))      # Train the model
```

```
  ep  17/30  loss 0.2961 acc 0.9085 | val acc 0.8588 auc 0.9405  64s
  ep  18/30  loss 0.2946 acc 0.9090 | val acc 0.8674 auc 0.9472  64s
  ep  19/30  loss 0.2910 acc 0.9123 | val acc 0.8765 auc 0.9420  64s
  ep  20/30  loss 0.2885 acc 0.9132 | val acc 0.8755 auc 0.9430  63s
  ep  21/30  loss 0.2857 acc 0.9143 | val acc 0.8767 auc 0.9440  63s
  ep  22/30  loss 0.2842 acc 0.9163 | val acc 0.8611 auc 0.9399  64s
  ep  23/30  loss 0.2829 acc 0.9159 | val acc 0.8729 auc 0.9416  64s
  ep  24/30  loss 0.2808 acc 0.9175 | val acc 0.8782 auc 0.9484  63s
  ep  25/30  loss 0.2807 acc 0.9182 | val acc 0.8824 auc 0.9505  63s
  ep  26/30  loss 0.2787 acc 0.9181 | val acc 0.8767 auc 0.9472  62s
  ep  27/30  loss 0.2759 acc 0.9207 | val acc 0.8711 auc 0.9515  62s
  ep  28/30  loss 0.2757 acc 0.9209 | val acc 0.8828 auc 0.9564  62s
  ep  29/30  loss 0.2732 acc 0.9219 | val acc 0.8848 auc 0.9540  62s
  ep  30/30  loss 0.2729 acc 0.9216 | val acc 0.8806 auc 0.9498  62s
best epoch 28: val AUC 0.9564
```

Figure 31

Code Snippet for Creating and Training the Hybrid CNN-BiLSTM Model

```
torch.manual_seed(42)                                                                                  # Fix a seed
model_hybrid = CNNBiLSTM(dropout=0.3, block_dropout=0.05, hidden=256, num_layers=1, head="centre")     # Create the hybrid model
run_hybrid = train(model_hybrid, "cnn_bilstm", (xtr, ytr), (xva, yva))                                 # Train the model
```

```
  ep  17/30  loss 0.2523 acc 0.9336 | val acc 0.8961 auc 0.9601  67s
  ep  18/30  loss 0.2490 acc 0.9349 | val acc 0.8971 auc 0.9582  68s
  ep  19/30  loss 0.2477 acc 0.9360 | val acc 0.8871 auc 0.9530  70s
  ep  20/30  loss 0.2458 acc 0.9366 | val acc 0.9100 auc 0.9686  74s
  ep  21/30  loss 0.2456 acc 0.9368 | val acc 0.9036 auc 0.9633  73s
  ep  22/30  loss 0.2419 acc 0.9390 | val acc 0.8917 auc 0.9583  69s
  ep  23/30  loss 0.2398 acc 0.9401 | val acc 0.8920 auc 0.9568  69s
  ep  24/30  loss 0.2385 acc 0.9417 | val acc 0.8833 auc 0.9589  74s
  ep  25/30  loss 0.2379 acc 0.9410 | val acc 0.8985 auc 0.9577  73s
  ep  26/30  loss 0.2368 acc 0.9422 | val acc 0.9104 auc 0.9678  73s
  ep  27/30  loss 0.2331 acc 0.9442 | val acc 0.8981 auc 0.9683  73s
  ep  28/30  loss 0.2343 acc 0.9427 | val acc 0.8989 auc 0.9587  73s
  ep  29/30  loss 0.2317 acc 0.9445 | val acc 0.9031 auc 0.9592  72s
  ep  30/30  loss 0.2315 acc 0.9452 | val acc 0.8951 auc 0.9571  71s
best epoch 20: val AUC 0.9686
```

Comparison of Validation Results and Learning Curves of Models

The validation results of the baseline CNN and hybrid CNN-BiLSTM models are displayed in Figure 32, while the learning curves of both models across 30 epochs are shown in Figure 33. Based on the validation results, the hybrid CNN-BiLSTM outperforms the baseline CNN model in terms of accuracy, ROC-AUC, and F1 score, suggesting that the recurrent component may contribute to the predictive capability of the model. On the other hand, despite its higher specificity, the baseline CNN suffers from a considerably lower sensitivity, making it less suitable for metastasis detection, where failing to identify the presence of tumour tissue can lead to delayed treatment and jeopardise patients' health (Hanna et al., 2020). In terms of the learning curves, the performance of both models improves steadily throughout the training process, as shown by the consistent decrease in training loss and the steady increase in training accuracy of both models across epochs. The validation loss and accuracy of both models also follow a similar trend, yet they fluctuate considerably between epochs. After approximately epoch 11, the validation accuracy of both models begins to plateau despite the continued increase in training accuracy, suggesting that both models are starting to overfit the training patches. Although the validation ROC-AUC of both models improves gradually, reaching the highest values of 0.9564 and 0.9686 at epochs 28 and 20 for the baseline CNN and the hybrid CNN-BiLSTM respectively, the risk of overfitting shall be further examined after performing hyperparameter tuning on both models.

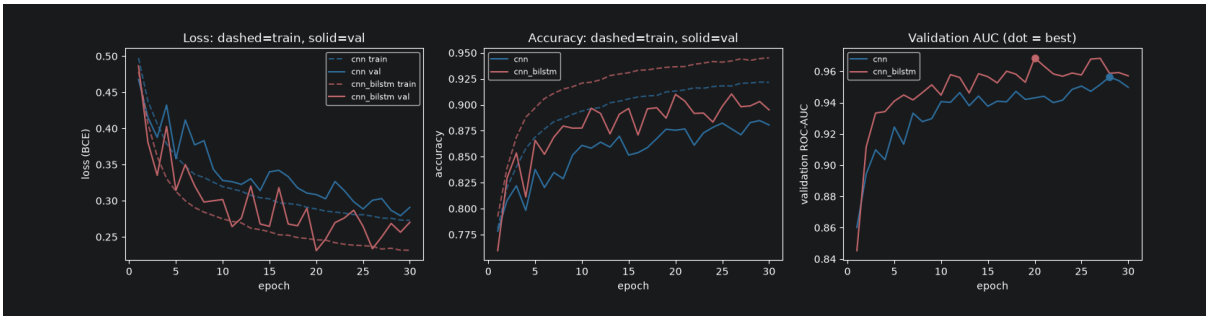
Figure 32

Validation Results of Baseline CNN and Hybrid CNN-BiLSTM before Tuning

	model	accuracy	auc	f1	mcc	sensitivity	specificity	nll
0	cnn	0.8828	0.9564	0.8751	0.7715	0.8212	0.9443	0.2867
1	cnn_bilstm	0.9100	0.9686	0.9091	0.8201	0.9005	0.9194	0.2309

Figure 33

Learning Curves of Baseline CNN and Hybrid CNN-BiLSTM before Tuning



Model Tuning

Tuning Configuration

Before tuning the baseline CNN and the hybrid CNN-BiLSTM models using random search, several configurations and functions are implemented in Figure 34, which include:

- **Dataset size for tuning**

  According to Wojciuk et al. (2024), using a reduced set of images when tuning the hyperparameters for CNN is able to reduce computational cost and time while yielding improved performance, provided that the subset maintains a balanced class distribution. Therefore, to improve computational feasibility of hyperparameter tuning, the training split is randomly reduced to 30,000 patches (approx. 43%) while the validation split is reduced to 8,192 patches (approx. 25%), each containing around 50% of positive patches.

- **`sample_params()` function**

  The `sample_params()` function randomly selects one value for each hyperparameter from the search space, producing a combination for each random search iteration.

- **`run_trial()` function**

  The `run_trial()` function is used to perform a single run of random search, where a new model is created with the selected model hyperparameters and trained on the sampled subsets using the sampled training hyperparameters. Once the run completes, the time taken, the hyperparameters used, and the best validation results of the trial are returned.

Figure 34

Code Snippet for Preparing the Tuning Subsets and Defining Helper Functions

```
rng = np.random.default_rng(42)                                          # Define the seed
SEARCH_TRAIN_N, SEARCH_VAL_N = 30_000, 8_192                             # Set the sampled dataset for hyperparameter tuning
tr_sub = np.sort(rng.choice(len(ytr), SEARCH_TRAIN_N, replace=False))
va_sub = np.sort(rng.choice(len(yva), SEARCH_VAL_N, replace=False))
xtr_s, ytr_s = np.ascontiguousarray(xtr[tr_sub]), ytr[tr_sub]            # Retrieve the image data and labels
xva_s, yva_s = np.ascontiguousarray(xva[va_sub]), yva[va_sub]

def sample_params(space, rng):                                           # Randomly select hyperparameters for tuning
    out = {}
    for key, dist in space.items():                                      # Loop through each hyperparameter
        if dist[0] == "log":                                             # Log-sampling values
            out[key] = float(np.exp(rng.uniform(np.log(dist[1]), np.log(dist[2]))))  # Select a random value
        elif dist[0] == "uniform":                                       # Values within a range
            out[key] = float(rng.uniform(dist[1], dist[2]))              # Select a random value
        else:
            out[key] = dist[1][int(rng.integers(len(dist[1])))]          # Select a random choice
    return out

def run_trial(key, params, tag):                                         # Run a trial with a hyperparameter configuration
    model_kwargs = {k: v for k, v in params.items() if k in MODEL_KEYS}  # Get parameters for model architecture
    train_kwargs = {k: v for k, v in params.items() if k not in MODEL_KEYS}  # Get parameters for training
    t0 = time.time()                                                     # Set the starting time
    torch.manual_seed(42)                                                # Set a seed
    model = MODELS[key]["cls"](**model_kwargs)                           # Create a new model
    rec = train(model, tag, (xtr_s, ytr_s), (xva_s, yva_s),              # Train the model
                epochs=SEARCH_EPOCHS, verbose=False, **train_kwargs)
    return {"model": key, "params": params, "seconds": time.time() - t0, **rec["best"]}  # Return trial result
```

Search Space

The search space used to perform random search is outlined in Table 2, together with its code implementation shown in Figure 35. Two types of hyperparameters are involved in the tuning process, namely the model hyperparameters like dropout (`dropout`) and the number of BiLSTM layers (`num_layers`) that define the architecture of the model, as well as the training hyperparameters like learning rate (`lr`) and batch size (`batch_size`) that control how each model learns.

Figure 35

Code Snippet for Defining the Hyperparameter Search Space

```
# Settings for hyperparameter search
N_TRIALS      = 20                          # 20 different configurations
SEARCH_EPOCHS = 6                           # 6 epochs to be trained

# Model architecture
MODELS = {
    "cnn":  {"label": "CNN",                # Model name
             "cls": BaselineCNN,            # Model class
             "model": model_baseline,       # Baseline model created
             "run": run_baseline},          # Results from baseline

    "cnn_bilstm": {"label": "CNN-BiLSTM",   # Model name
                   "cls": CNNBiLSTM,        # Model class
                   "model": model_hybrid,   # Baseline hybrid model
                   "run": run_hybrid},      # Results from hybrid
}

# Hyperparameters to tune model architecture
MODEL_KEYS = {"widths", "dropout", "block_dropout", "head", "fc_dim", "hidden", "num_layers", "grid"}

# Search space for hyperparameter tuning
SPACES = {
    "cnn": {
        "lr":              ("log", 1e-4, 3e-3),
        "weight_decay":    ("log", 1e-6, 1e-2),
        "dropout":         ("uniform", 0.10, 0.50),
        "block_dropout":   ("choice", [0.0, 0.05, 0.10]),
        "head":            ("choice", ["gap", "centre", "both"]),
        "fc_dim":          ("choice", [128, 256]),
        "label_smoothing": ("choice", [0.0, 0.05, 0.10]),
        "aug_hed_sigma":   ("choice", [0.0, 0.03, 0.05, 0.10]),
        "batch_size":      ("choice", [128, 256]),
    },
    "cnn_bilstm": {
        "lr":              ("log", 1e-4, 3e-3),
        "weight_decay":    ("log", 1e-6, 1e-2),
        "dropout":         ("uniform", 0.10, 0.50),
        "block_dropout":   ("choice", [0.0, 0.05, 0.10]),
        "head":            ("choice", ["attn", "centre", "both"]),
        "hidden":          ("choice", [128, 192, 256]),
        "num_layers":      ("choice", [1, 2]),
        "fc_dim":          ("choice", [128, 256]),
        "label_smoothing": ("choice", [0.0, 0.05, 0.10]),
        "aug_hed_sigma":   ("choice", [0.0, 0.03, 0.05, 0.10]),
        "batch_size":      ("choice", [128, 256]),
    },
}
```

Table 2

Search Space for Random Search

Hyperparameter	Type	Values	Model	Description
Learning rate `lr`	Training	10^{-4} to 3×10^{-3} (log scale)	Both	Size of each weight update step
Weight decay `weight_decay`	Training	10^{-6} to 10^{-2} (log scale)	Both	Penalty on large weights to reduce overfitting
Batch size `batch_size`	Training	• 128 • 256	Both	Number of patches used in each weight update
Label smoothing `label_smoothing`	Training	• 0.00 • 0.05 • 0.10	Both	How far the labels are moved away from 0 and 1 to reduce overconfidence
Stain jitter strength `aug_hed_sigma`	Training	• 0.00 • 0.03 • 0.05 • 0.10	Both	Strength of the random H&E stain colour changes during augmentation
Dropout `dropout`	Model	0.1 to 0.5 (uniform)	Both	Fraction of features that are randomly ignored in the dense layer, and between BiLSTM layers when two are stacked
Block dropout `block_dropout`	Model	• 0.00 • 0.05 • 0.10	Both	Fraction of feature maps that are randomly ignored after each convolutional block
Dense layer size `fc_dim`	Model	• 128 • 256	Both	Number of neurons in the dense layer before the output
Read-out `head`	Model	CNN • gap • centre • both CNN-BiLSTM • attn • centre • both	Both	The part of the final features is used for the prediction (the entire image or centre 32×32 region)
BiLSTM hidden size `hidden`	Model	• 128 • 192 • 256	CNN-BiLSTM	Number of memory units in each direction of BiLSTM
BiLSTM layers `num_layers`	Model	• 1 • 2	CNN-BiLSTM	Number of stacked BiLSTM layers

Random Search

With the configurations established and the search space defined, the random search technique is implemented in Figure 36 to identify the best-performing combination of hyperparameters for both the baseline CNN and the hybrid CNN-BiLSTM models. For this project, random search is preferred over grid search due to the reason that the search space is too large to be evaluated exhaustively, where the discrete hyperparameters alone produce 432 combinations for the baseline CNN and 2,592 combinations for the hybrid CNN-BiLSTM model. This also aligns with Vincent and Jidesh (2023), who outlined that the random search technique is more efficient in image classification tasks that involve high-dimensional search spaces compared with the grid search technique that easily suffers from the curse of dimensionality. Therefore, the random search technique is applied to generate 20 random hyperparameter combinations for each model using the `sample_params()` function, then each combination is trained for 6 epochs on the sampled subsets using `run_trial()` to obtain its best validation results.

Figure 36

Code Snippet to Perform Random Search on Baseline CNN and Hybrid CNN-BiLSTM

```
# Store results from all trials and search time
random_trials, search_time = {}, {}

# Loop through each hyperparameter
for key in MODELS:
    rng = np.random.default_rng(42)                                                    # Set a seed
    trials, t_start = [], time.time()                                                  # Record start time
    for i in range(1, N_TRIALS + 1):                                                   # Run 20 trials
        rec = run_trial(key, sample_params(SPACES[key], rng), f"{key}_rand_{i:03d}")   # Perform training
        trials.append(rec)                                                             # Save results of each trial
    random_trials[key] = trials                                                        # Save results of all trials
    search_time[key] = time.time() - t_start                                           # Save ending time

# Show random search results in table
display(pd.DataFrame([{"model": MODELS[t["model"]]["label"], **t["params"], "val_auc": t["val_auc"], "val_acc": t["val_acc"]} for trials in random_trials.values() for t in trials])
        .sort_values(["model", "val_auc"], ascending=[True, False]).round(5))
```

	model	lr	weight_decay	dropout	block_dropout	head	fc_dim	label_smoothing	aug_hed_sigma	batch_size	val_auc	val_acc	hidden	num_layers
2	CNN	0.00089	0.00195	0.27757	0.05	gap	128	0.05	0.10	128	0.93295	0.83358	NaN	NaN
17	CNN	0.00025	0.00753	0.41150	0.00	both	256	0.05	0.05	128	0.93287	0.85608	NaN	NaN
7	CNN	0.00108	0.00002	0.43298	0.05	both	256	0.05	0.10	128	0.93218	0.84937	NaN	NaN
10	CNN	0.00897	0.00008	0.32689	0.00	both	256	0.05	0.05	256	0.93132	0.81592	NaN	NaN
4	CNN	0.00141	0.00001	0.28669	0.05	gap	256	0.00	0.05	256	0.92980	0.84229	NaN	NaN
0	CNN	0.00139	0.00006	0.44344	0.00	both	128	0.00	0.05	256	0.92900	0.85510	NaN	NaN
16	CNN	0.00034	0.00008	0.14720	0.05	both	128	0.10	0.03	256	0.92394	0.84094	NaN	NaN
11	CNN	0.00067	0.00002	0.11233	0.05	centre	256	0.00	0.03	128	0.92272	0.83875	NaN	NaN
12	CNN	0.00182	0.00001	0.12332	0.10	gap	256	0.00	0.03	256	0.91765	0.83289	NaN	NaN
13	CNN	0.00866	0.00137	0.36573	0.05	centre	128	0.10	0.03	128	0.91561	0.83374	NaN	NaN
8	CNN	0.00102	0.00000	0.17996	0.10	gap	256	0.10	0.10	256	0.91436	0.83289	NaN	NaN
5	CNN	0.00126	0.00741	0.23033	0.10	centre	128	0.05	0.10	128	0.91374	0.83044	NaN	NaN
9	CNN	0.00110	0.00133	0.28357	0.05	centre	128	0.00	0.00	128	0.91050	0.81738	NaN	NaN
18	CNN	0.00014	0.00408	0.28231	0.10	gap	256	0.00	0.10	256	0.91050	0.82458	NaN	NaN
19	CNN	0.00018	0.00267	0.40341	0.05	both	256	0.05	0.03	256	0.90852	0.81555	NaN	NaN
3	CNN	0.00167	0.00034	0.40324	0.10	centre	128	0.10	0.03	256	0.90751	0.81812	NaN	NaN
15	CNN	0.00017	0.00061	0.27846	0.00	centre	128	0.00	0.05	256	0.90561	0.81868	NaN	NaN
6	CNN	0.00016	0.00008	0.19076	0.05	both	256	0.05	0.00	256	0.90387	0.81421	NaN	NaN
1	CNN	0.00133	0.00139	0.15125	0.10	centre	256	0.05	0.00	256	0.89905	0.80737	NaN	NaN
14	CNN	0.00011	0.00000	0.38894	0.10	centre	256	0.00	0.10	256	0.89653	0.80811	NaN	NaN
29	CNN-BiLSTM	0.00135	0.00035	0.32143	0.00	centre	256	0.00	0.03	128	0.94220	0.85608	256.0	1.0
33	CNN-BiLSTM	0.00107	0.00006	0.25241	0.00	attn	256	0.05	0.10	128	0.94142	0.86194	256.0	2.0
30	CNN-BiLSTM	0.00021	0.00004	0.44136	0.00	attn	256	0.00	0.10	128	0.93872	0.86035	256.0	1.0
38	CNN-BiLSTM	0.00031	0.00000	0.14136	0.00	centre	128	0.10	0.10	256	0.93799	0.86047	256.0	1.0
20	CNN-BiLSTM	0.00139	0.00006	0.44344	0.00	both	256	0.10	0.05	256	0.93746	0.85559	128.0	1.0
39	CNN-BiLSTM	0.00033	0.00023	0.10912	0.00	both	128	0.10	0.00	128	0.93725	0.84717	192.0	1.0
28	CNN-BiLSTM	0.00048	0.00019	0.15592	0.00	attn	256	0.05	0.10	256	0.93500	0.85986	192.0	2.0
25	CNN-BiLSTM	0.00019	0.00000	0.29028	0.00	attn	256	0.05	0.00	256	0.93068	0.85156	192.0	2.0
24	CNN-BiLSTM	0.00017	0.00054	0.39798	0.05	both	256	0.05	0.00	128	0.92914	0.84863	192.0	1.0
36	CNN-BiLSTM	0.00028	0.00021	0.17071	0.05	both	128	0.10	0.05	128	0.92891	0.84741	128.0	2.0
37	CNN-BiLSTM	0.00084	0.00022	0.35994	0.05	attn	256	0.00	0.00	128	0.92827	0.83362	256.0	1.0
26	CNN-BiLSTM	0.00108	0.00002	0.43298	0.05	both	256	0.00	0.00	256	0.92701	0.83801	256.0	1.0
35	CNN-BiLSTM	0.00115	0.00006	0.20890	0.00	attn	128	0.05	0.05	128	0.92432	0.82593	192.0	2.0
21	CNN-BiLSTM	0.00145	0.00000	0.28015	0.05	centre	256	0.05	0.03	256	0.92250	0.81519	128.0	2.0
23	CNN-BiLSTM	0.00033	0.00763	0.45725	0.10	both	128	0.05	0.03	128	0.92236	0.84167	256.0	1.0
31	CNN-BiLSTM	0.00095	0.00017	0.41356	0.10	centre	128	0.10	0.03	128	0.91836	0.83337	192.0	1.0
22	CNN-BiLSTM	0.00045	0.00001	0.32183	0.10	attn	128	0.05	0.00	256	0.91213	0.80176	256.0	2.0
27	CNN-BiLSTM	0.00016	0.00001	0.10294	0.10	both	128	0.10	0.03	256	0.90914	0.82251	256.0	2.0
34	CNN-BiLSTM	0.00015	0.00704	0.46343	0.05	both	256	0.10	0.03	256	0.90841	0.80798	192.0	1.0
32	CNN-BiLSTM	0.00011	0.00000	0.38894	0.10	centre	256	0.05	0.10	128	0.90242	0.80554	256.0	1.0

Best Hyperparameter Configurations and Influence of Hyperparameters

The best hyperparameter combinations for the baseline CNN and the hybrid CNN-BiLSTM models are shown in Figure 37, while Figure 38 shows the Spearman rank correlation between each hyperparameter and validation ROC-AUC. Table 3 further compares the hyperparameter configurations of the baseline and optimised models. Based on the results, it is observed that:

- **Lower validation ROC-AUC compared with the baseline runs**
  The best trials of the optimised CNN and CNN-BiLSTM models achieved validation ROC-AUC values of 0.9329 and 0.9422 respectively, which are lower than the ROC-AUC values of 0.9564 and 0.9686 obtained from the baseline runs in Figure 32, due to the reason that each trial is trained for only 6 epochs on the sampled subsets. Therefore, these values shall not be compared directly with the baseline runs until both models are retrained using the optimised hyperparameters and the full training and validation sets.

- **Similar training hyperparameters preferred by both models**
  Both optimised models prefer a higher learning rate and a smaller batch size if compared to the baseline configuration, which is consistent with the directions of their weak-to-moderate correlations in Figure 38, where the learning rate is positively correlated and the batch size is negatively correlated with validation ROC-AUC for both models. As each trial is limited to only 6 epochs, a higher learning rate allows models to take larger update steps and converge more quickly. Meanwhile, the smaller batch size doubles the number of weight updates in each epoch, allowing the neural network models to make more progress within the limited number of epochs to improve their performance.

- **Block dropout as the most influential hyperparameter**
  Block dropout shows the highest negative correlation with validation ROC-AUC among all hyperparameters, where it has a Spearman correlation of -0.49 for the baseline CNN and -0.85 for the hybrid CNN-BiLSTM. This indicates that trials that are configured with a lower block dropout generally achieved better performance, as reflected in the optimised CNN-BiLSTM model that has the block dropout removed entirely. However, the lack of block dropout indicates that no feature maps will be randomly dropped during the training process. Although this allows models to learn more quickly within a limited number of epochs, this also removes a source of regularisation, which may cause models to heavily rely on specific features and increase the risk of overfitting when fully trained.

Figure 37

Best Hyperparameter Configurations Obtained from Random Search

```
CNN (best validation AUC: 0.9329)
  lr               0.000893
  weight_decay     0.001955
  dropout          0.277366
  block_dropout    0.05
  head             gap
  fc_dim           128
  label_smoothing  0.05
  aug_hed_sigma    0.1
  batch_size       128

CNN-BiLSTM (best validation AUC: 0.9422)
  lr               0.001349
  weight_decay     0.000346
  dropout          0.321432
  block_dropout    0.0
  head             centre
  hidden           256
  num_layers       1
  fc_dim           256
  label_smoothing  0.0
  aug_hed_sigma    0.03
  batch_size       128
```

Figure 38

Spearman Rank Correlation Between Hyperparameters and Validation ROC-AUC

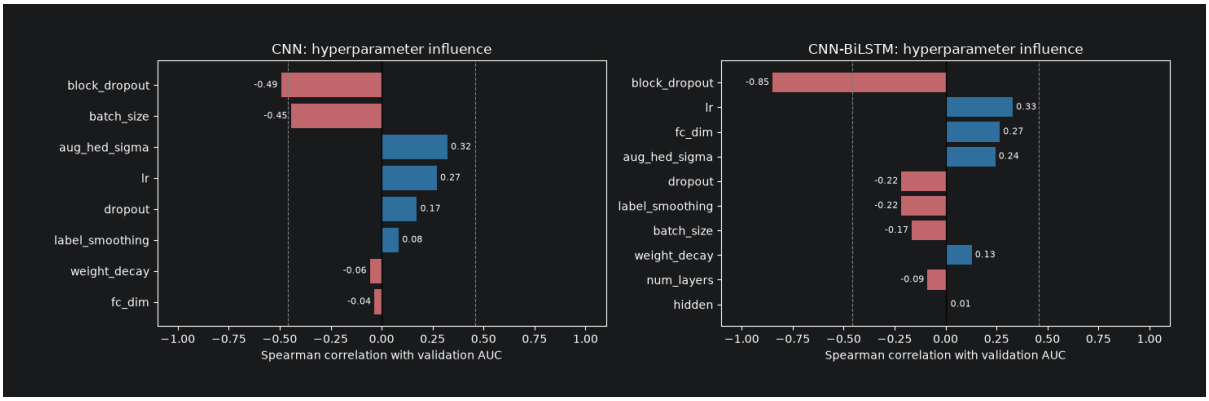


Table 3

Comparison of Baseline and Optimised Hyperparameters

Hyperparameter	Baseline Run	Optimised CNN	Optimised CNN-BiLSTM
Learning rate	3×10^{-4}	8.93×10^{-4}	1.35×10^{-3}
Weight decay	10^{-4}	1.96×10^{-3}	3.46×10^{-4}
Batch size	256	128	128
Label smoothing	0.05	0.05	0.00
Stain jitter strength	0.05	0.10	0.03
Dropout	0.30	0.28	0.32
Block dropout	0.05	0.05	0.00
Dense layer size	128	128	256
Read-out (head)	centre	gap	centre
BiLSTM hidden size	256	-	256
BiLSTM layers	1	-	1

Retraining of Optimised Models

Both the baseline CNN and the hybrid CNN-BiLSTM models are retrained using their best-performing hyperparameters on the 70,000-patch stratified training set and the full validation set for 30 epochs, as shown in Figure 39, together with the corresponding performance metrics.

Figure 39

Code Snippet to Retrain Models with Best-Performing Hyperparameters

```
# Dictionary to store tuned models
tuned_models, tuned_runs = {}, {}

# Loop through each model
for key in MODELS:

    # Extract the best hyperparameters
    params = best_cfg[key]["params"]

    # Separate into model hyperparameters and training hyperparameters
    model_kwargs = {k: v for k, v in params.items() if k in MODEL_KEYS}
    train_kwargs = {k: v for k, v in params.items() if k not in MODEL_KEYS}

    # Set the seed
    torch.manual_seed(42)

    # Create the model with the model hyperparameters
    model = MODELS[key]["cls"](**model_kwargs)

    # Train the model with the training hyperparameters
    tuned_runs[key] = train(model, f"{key}_tuned", (xtr, ytr), (xva, yva), **train_kwargs)
    tuned_models[key] = model

# Retrieve the necessary information
rows = []
for key in MODELS:
    probs, tgt = predict(tuned_models[key], xva, yva)           # Convert score into probabilities
    m = metrics(probs, tgt)                                     # Retrieve the evaluation metrics
    rows.append({"model": MODELS[key]["label"],
                 "accuracy": m["accuracy"],
                 "auc": m["auc"],
                 "f1": m["f1"],
                 "mcc": m["mcc"],
                 "sensitivity": m["sensitivity"],
                 "specificity": m["specificity"],
                 "nll": m["nll"]})

# Output results as a table
display(pd.DataFrame(rows).round(4))
```

```
  ep  17/30  loss 0.1560 acc 0.9426 | val acc 0.8613 auc 0.9505  67s
  ep  18/30  loss 0.1517 acc 0.9434 | val acc 0.8745 auc 0.9578  65s
  ep  19/30  loss 0.1495 acc 0.9455 | val acc 0.8365 auc 0.9500  65s
  ep  20/30  loss 0.1478 acc 0.9460 | val acc 0.8918 auc 0.9590  65s
  ep  21/30  loss 0.1445 acc 0.9468 | val acc 0.8737 auc 0.9694  65s
  ep  22/30  loss 0.1427 acc 0.9474 | val acc 0.8834 auc 0.9574  67s
  ep  23/30  loss 0.1417 acc 0.9474 | val acc 0.8415 auc 0.9479  64s
  ep  24/30  loss 0.1371 acc 0.9506 | val acc 0.9105 auc 0.9683  64s
  ep  25/30  loss 0.1362 acc 0.9500 | val acc 0.9081 auc 0.9650  65s
  ep  26/30  loss 0.1333 acc 0.9523 | val acc 0.9108 auc 0.9671  65s
  ep  27/30  loss 0.1308 acc 0.9531 | val acc 0.8776 auc 0.9551  65s
  ep  28/30  loss 0.1306 acc 0.9527 | val acc 0.9007 auc 0.9632  65s
  ep  29/30  loss 0.1286 acc 0.9535 | val acc 0.8944 auc 0.9611  64s
  ep  30/30  loss 0.1257 acc 0.9546 | val acc 0.9075 auc 0.9671  65s
best epoch 21: val AUC 0.9694
```

	model	accuracy	auc	f1	mcc	sensitivity	specificity	nll
0	CNN	0.9064	0.9648	0.9045	0.8134	0.8877	0.9251	0.2407
1	CNN-BiLSTM	0.8737	0.9694	0.8588	0.7645	0.7686	0.9787	0.3007

Model Evaluation

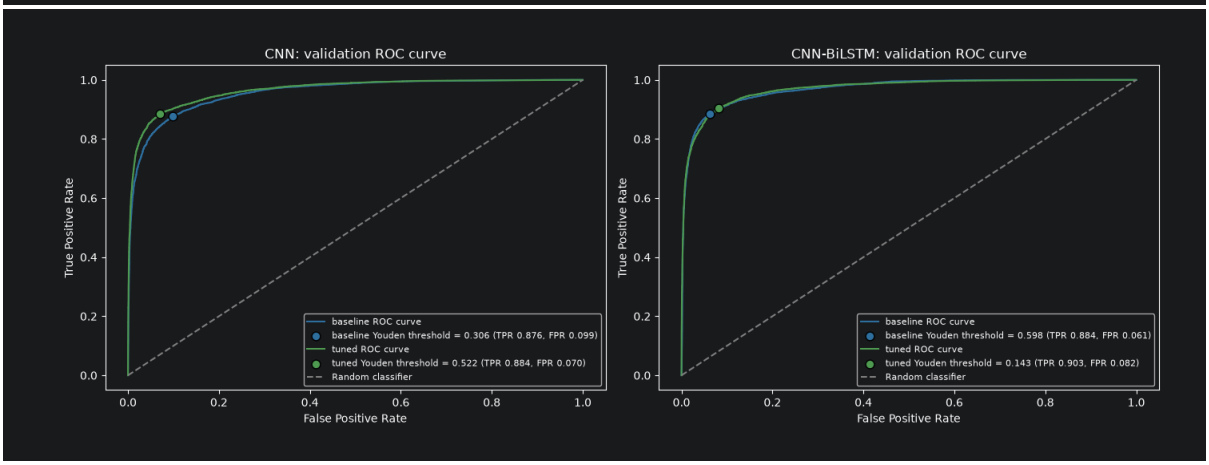
Determining the Decision Threshold

Before evaluating the trained models on the test set, the decision threshold used to classify the presence of metastatic tissues must first be determined. As the default threshold of 0.5 does not guarantee the best balance between sensitivity and specificity for each model, Youden's J statistic is applied to identify the decision threshold that maximises the difference between the true positive rate (TPR) and the false positive rate (FPR) for each model (Hassanzad & Hajian-Tilaki, 2024). As shown in Figure 40, the thresholds are determined using only the validation dataset to prevent the test split from influencing decisions, thereby ensuring a fair performance measurement. Based on the calculations, the Youden threshold of each model is marked at the point furthest above the diagonal line of a random classifier on the ROC curves, where the obtained threshold will be applied when evaluating each model on the test set subsequently.

Figure 40

Code Snippet to Calculate and Plot the Youden Threshold for Each Model

```
# Calculate the Youden threshold for each model
thresholds = {}
fig, axes = plt.subplots(1, len(MODELS), figsize=(7 * len(MODELS), 5))
for ax, key in zip(axes, MODELS):
    for stage, model, name, c in (("baseline", MODELS[key]["model"], key, "#2E6F9E"),
                                  ("tuned", tuned_models[key], f"{key}_tuned", "#4E9A51")):
        probs_val, tgt = predict(model, xva, yva)                                  # Validation probabilities
        fpr, tpr, thr = roc_curve(tgt, probs_val)                                  # ROC curve
        i = int(np.argmax(tpr - fpr))                                              # Youden point
        thresholds[name] = float(thr[i])                                           # Store threshold
        ax.plot(fpr, tpr, color=c, label=f"{stage} ROC curve")                     # Plot the ROC curve
        ax.scatter(fpr[i], tpr[i], s=60, zorder=5, color=c, edgecolor="black",
                   label=f"{stage} Youden threshold = {thr[i]:.3f} (TPR {tpr[i]:.3f}, FPR {fpr[i]:.3f})")
    ax.plot([0, 1], [0, 1], "--", color="grey", label="Random classifier")
    ax.set(xlabel="False Positive Rate", ylabel="True Positive Rate",
           title=f"{MODELS[key]['label']}: validation ROC curve")
    ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```



Evaluation Metrics

With the decision thresholds determined, several performance metrics for binary classification are applied to evaluate each model, as outlined in Table 4 with their corresponding descriptions. Threshold-based metrics like accuracy, sensitivity and F1 score measure the classification performance of the models, ROC-AUC measures their ability to discriminate between normal and tumour patches, while negative log-likelihood and Brier score evaluate the calibration of predicted probabilities for each model.

Table 4

Evaluation Metrics Used to Assess Model Performance

Category	Metric	Description
Threshold-based	Accuracy	Proportion of all patches that are correctly classified
	Sensitivity	Proportion of tumour patches that are correctly detected, reflecting the ability of models to avoid missed metastases
	Specificity	Proportion of normal patches that are correctly identified, reflecting the ability of models to avoid false alarms
	Precision	Proportion of patches predicted as tumour that are actually tumour
	F1 score	Harmonic mean of precision and sensitivity, balancing missed and false detections
	Matthews correlation coefficient (MCC)	Correlation between predicted and actual labels using all four cells of the confusion matrix, ranging from −1 to 1
Discrimination	ROC-AUC	Probability that a randomly selected tumour patch receives a higher predicted probability than a randomly selected normal patch, independent of the decision threshold
Calibration	Negative log-likelihood (NLL)	Average penalty on predicted probabilities, where confident but incorrect predictions are penalised heavily
	Brier score	Mean squared difference between predicted probabilities and actual labels, where lower values indicate better calibration

Performance Comparison of Baseline and Optimised Models

To evaluate the performance of the optimised models, both their learning progress and their performance metrics are examined. The learning curves of the baseline runs and the optimised models are shown in Figure 41 to understand how the optimised hyperparameters affect the training process, where the accuracy curve is computed using the default decision threshold of 0.5. Additionally, the performance metrics of baseline and optimised models on the validation set, which are measured at their respective Youden thresholds, are also shown in Figure 42, allowing all four models to be compared based on their best validation results so that the champion model can be determined.

Figure 41

Learning Curves of the Baseline and Optimised CNN and CNN-BiLSTM Models

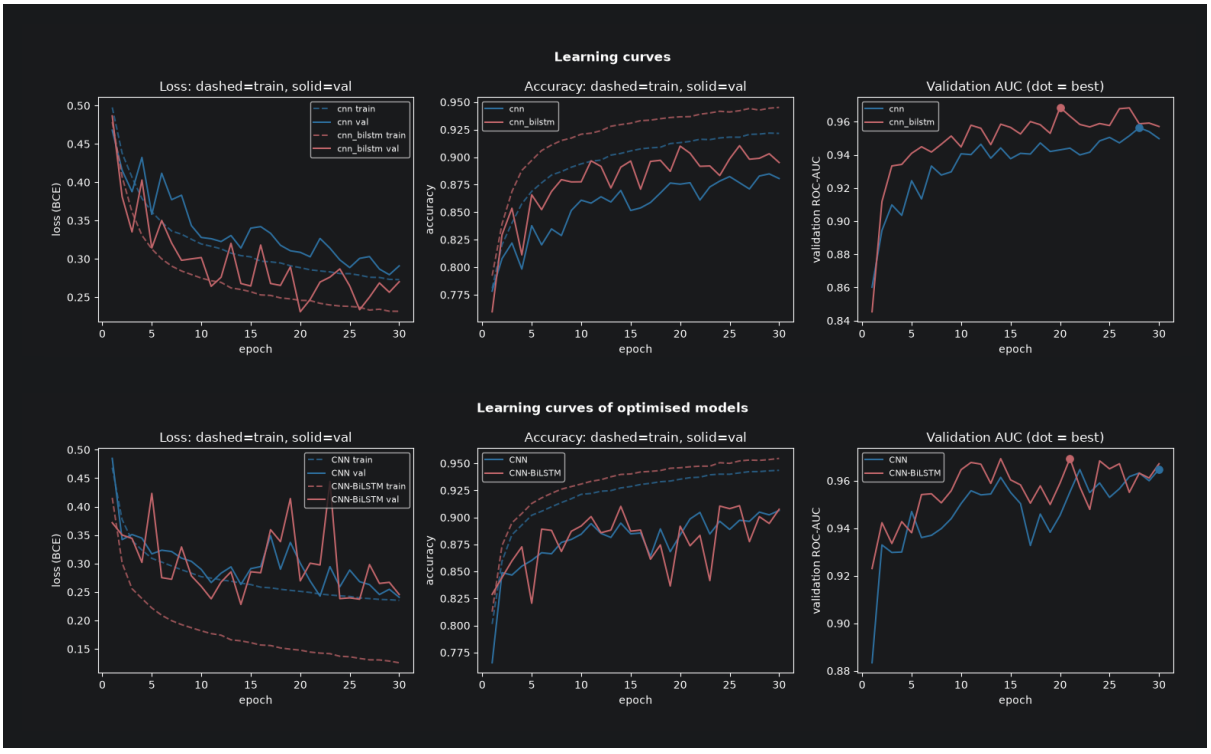


Figure 42

Validation Performance Metrics of Models at Their Respective Youden Thresholds

model	threshold	accuracy	auc	f1	mcc	sensitivity	specificity	nll	parameters
CNN (baseline)	0.3058	0.8887	0.9564	0.8872	0.7776	0.8761	0.9013	0.2867	1206497
CNN (optimised)	0.5219	0.9070	0.9648	0.9047	0.8148	0.8840	0.9299	0.2407	1206497
CNN-BiLSTM (baseline)	0.5979	0.9113	0.9686	0.9088	0.8238	0.8840	0.9385	0.2309	2291937
CNN-BiLSTM (optimised)	0.1433	0.9108	0.9694	0.9100	0.8216	0.9033	0.9182	0.3007	2357985

Some key observations derived from the results include:

- **Optimised CNN-BiLSTM fits the training data more closely**

  As shown in Figure 41, the optimised CNN-BiLSTM model achieves a training loss below 0.15 while its training accuracy exceeds 95%, which is respectively lower and slightly higher than the results obtained from its baseline run. However, the optimised hybrid model does not implement any label smoothing or block dropout mechanisms after its hyperparameter tuning, indicating that the model has the potential to become more aggressive when fitting the training patches, which allows it to achieve a significant decrease in its training loss and improve its training accuracy but not necessarily contribute to model learning (Salehin & Kang, 2023). In fact, the gap between the training and validation curves of the optimised hybrid model begins to widen after it iterates through the first few epochs, thereby reflecting that the model begins to overfit on the training patches instead of picking up useful features to improve its model performance and generalisability.

- **Unstable validation performance of the optimised CNN-BiLSTM**

  It is observed from Figure 41 that the validation loss and accuracy of the optimised CNN-BiLSTM also fluctuates considerably across epochs, where sharp spikes are observed at epochs 5, 19 and 23 on both the loss and accuracy curves. Such an instability can be attributed to the tuned hyperparameters configuration that increased the learning rate and reduced the batch size of the hybrid model. Although these adjustments are able to allow the optimised hybrid model to learn more effectively within the first few epochs of each tuning trial, when the model is fully trained for 30 epochs on the larger training and validation sets, the higher learning rate increases the tendency of the model to overshoot during its weight updates, while the smaller batch size introduces additional noise into each update (Sclocchi & Wyart, 2024). Coupled with the removal of block dropout and label smoothing which weakens the regularisation imposed on the optimised hybrid model, this further deteriorates the model's learning performance as the training progress becomes further destabilised (Liang et al., 2024). As a result, the optimised hybrid CNN-BiLSTM model suffers from both unstable learning and overfitting issues, despite its capability to achieve the highest validation ROC-AUC value of 0.9694 among all models.

- **Hyperparameter tuning improves the CNN but not the CNN-BiLSTM**

  From Figure 42, the hyperparameter tuning process improves the performance of CNN across all metrics, where its accuracy increases from 0.8887 to 0.9070 and its negative log-likelihood decreases from 0.2867 to 0.2407, making its performance comparable to the baseline CNN-BiLSTM. However, in terms of CNN-BiLSTM, hyperparameter tuning does not yield noticeable benefits for the model. Despite its slight increase in ROC-AUC from 0.9686 to 0.9694, sensitivity from 0.8840 to 0.9033, and F1 score from 0.9088 to 0.9100, the CNN-BiLSTM model experiences a decline in other performance metrics, with its accuracy decreasing from 0.9113 to 0.9108, its specificity reducing from 0.9385 to 0.9182, and its negative log-likelihood increasing from 0.2309 to 0.3007. This reflects a decline in the calibration of the optimised model, which aligns with the overfitting issue observed from its learning curves in Figure 41.

- **Optimised CNN-BiLSTM has a considerably lower Youden threshold**

  It is also observed from Figure 42 that the Youden threshold of the optimised CNN-BiLSTM model is 0.1433, which is far lower than other models that range from 0.3058 to 0.5979. Although the model achieves the highest ROC-AUC of 0.9694, indicating that the model is still able to distinguish between tumour and normal patches, the low Youden threshold reflects that any patch with a predicted probability above 14.33% is classified as tumour. At this threshold, the specificity of the optimised model decreases from 0.9385 to 0.9182 compared with its baseline. Coupled with its negative log-likelihood of 0.3007, which is the highest among all models, this indicates that the predicted probabilities of the model are poorly calibrated and do not reflect the actual likelihood of metastasis, limiting the clinical usefulness and trustworthiness of the model when it is implemented in the medical field to support risk-based decisions (Faghani et al., 2023).

- **Marginal gain of the CNN-BiLSTM at twice the model size**

  Although the optimised CNN-BiLSTM model achieves better performance than the optimised CNN in terms of ROC-AUC (0.9694 vs 0.9648), accuracy (0.9108 vs 0.9070) and sensitivity (0.9033 vs 0.8840), these improvements are accompanied with the cost of increased complexity. Based on Figure 42, the optimised CNN-BiLSTM model contains 2,357,985 parameters, approximately twice the 1,206,497 parameters of the CNN model. This indicates that the recurrent component of the hybrid model only provides slight improvements in predictive performance but requires a substantial increase in model size.

Champion Model Selection

Based on the findings from Figures 41 and 42, the champion model is selected using the validation results of the four models. To align with existing literature, the validation ROC-AUC is selected as the primary selection criterion. As shown in Figure 43, models with ROC-AUC falling within 0.001 of the highest value are considered tied, which the tie will be broken using negative log-likelihood as the secondary criterion.

Under this rule, both the baseline and optimised CNN-BiLSTM models are tied, as they achieved an ROC-AUC value of 0.9686 and 0.9694 respectively. However, the optimised CNN-BiLSTM model suffers from unstable learning, while its low Youden threshold and high negative log-likelihood indicate that its predictions are poorly calibrated. Therefore, the baseline CNN-BiLSTM model is selected as the champion model instead, not only due to its lower negative log-likelihood (0.2309 vs 0.3007), but also due to its performance as the model is able to outperform both CNN models across all evaluation metrics except sensitivity, where the baseline CNN-BiLSTM obtains the same value as the optimised CNN. Although the baseline CNN-BiLSTM model requires a larger model size, its better predictive capability makes it preferable over the optimised CNN, and it is applied to the test set in Figure 44 to evaluate its performance on unseen data.

Figure 43

Code Snippet for Selecting the Champion Model Based on Validation Results

```
tied = val_df[val_df["auc"] >= val_df["auc"].max() - 0.001]          # Models with ROC-AUC within 0.001 are tied
BEST_KEY = tied["nll"].idxmin()                                      # The tie is broken using negative log-likelihood
champion = candidates[BEST_KEY]["model"]                             # Retrieve the details of the champion model
champion_label = candidates[BEST_KEY]["label"]
thr_champ = thresholds[BEST_KEY]
```

Figure 44

Code Snippet for Evaluating the Champion Model on the Test Set

```
xte = np.load(PROCESSED / "test_full_x.npy", mmap_mode="r")                    # Load the test set
yte = np.load(PROCESSED / "test_full_y.npy").astype(np.float32).reshape(-1)
test_probs, y_test = predict(champion, xte, yte)                               # Predict using the champion model
m_test = metrics(test_probs, y_test, threshold=thr_champ)                      # Retrieve metrics
display(pd.DataFrame([{"model": champion_label, "threshold": thr_champ,        # Display results
                       **{k: m_test[k] for k in ("accuracy", "auc", "sensitivity", "specificity",
                                                 "precision", "f1", "mcc", "nll", "brier")}}]).round(4))
```

	model	threshold	accuracy	auc	sensitivity	specificity	precision	f1	mcc	nll	brier
0	CNN-BiLSTM (baseline)	0.5979	0.8764	0.9541	0.8091	0.9437	0.9349	0.8675	0.7598	0.2828	0.0855

Final Evaluation of Champion Model on Test Set

The champion model is evaluated on the test set from three perspectives:

- **Confusion Matrix**

  Using its Youden threshold of 0.5979, the baseline CNN-BiLSTM model is able to correctly classify 15,468 (94.4%) normal patches and 13,251 (80.9%) tumour patches, as shown in Figure 45. However, 923 (5.6%) normal patches are incorrectly classified as tumour, while 3,126 (19.1%) tumour patches are missed by the model. Compared with the validation set, these results indicate that the performance of the model decreases on the test set, particularly in terms of sensitivity that drops from 0.8840 to 0.8091. This raises concerns if the model is adopted for clinical use, as the increased false negative rate indicates that the model may misclassify tumour patches as normal tissue, which may lead to delayed treatment that endangers patient health (Evans & Snead, 2024).

- **ROC Curve**

  The ROC curve of the champion model on the test set is shown in Figure 46, achieving an ROC-AUC value of 0.9541. As the curve rises steeply towards the top-left corner and remains well above the diagonal line of a random classifier, this indicates that the model maintains a strong discriminative capability to distinguish between unseen normal and tumour patches. However, the test ROC-AUC value is slightly lower than the validation value of 0.9686, which indicates a small generalisation gap that could be addressed in future work with stronger regularisation or incorporating more diverse training data.

- **Calibration Curve**

  Figure 47 displays the calibration curve of the champion model on the test set, together with a histogram displaying the counts of its predicted probabilities. The histogram shows that most predictions made by the model are close to 0 or 1, indicating that the model is confident in the majority of its predictions. However, the calibration curve lies above the diagonal line for predicted probabilities between 0.1 to 0.7, which reflects the tendency of the model to underestimate the likelihood of tumour tissues in patches that it is uncertain about. In other words, the model tends to classify patches as normal when the model has a low confidence towards its predictions. This is consistent with the high number of false negatives in the confusion matrix, which could be mitigated in future work by recalibrating the model or selecting a sensitivity-oriented decision threshold.

Figure 45

Code Snippet for Confusion Matrix of the Champion CNN-BiLSTM Model on the Test Set

```
cm = np.array([[m_test["tn"], m_test["fp"]], [m_test["fn"], m_test["tp"]]])
cm_norm = cm / cm.sum(axis=1, keepdims=True)
fig, ax = plt.subplots(figsize=(5, 4.2))
ax.imshow(cm_norm, cmap="Blues", vmin=0, vmax=1)
for i in range(2):
    for j in range(2):
        ax.text(j, i, f"{cm[i, j]:,}\n({cm_norm[i, j]:.1%})", ha="center", va="center",
                color="white" if cm_norm[i, j] > 0.5 else "black", fontweight="bold")
ax.set(xticks=[0, 1], yticks=[0, 1], xticklabels=["Predicted normal", "Predicted tumour"],
       yticklabels=["Actual normal", "Actual tumour"],
       title=f"{champion_label}: confusion matrix (test set)")
ax.grid(False)
plt.tight_layout()
plt.show()
```

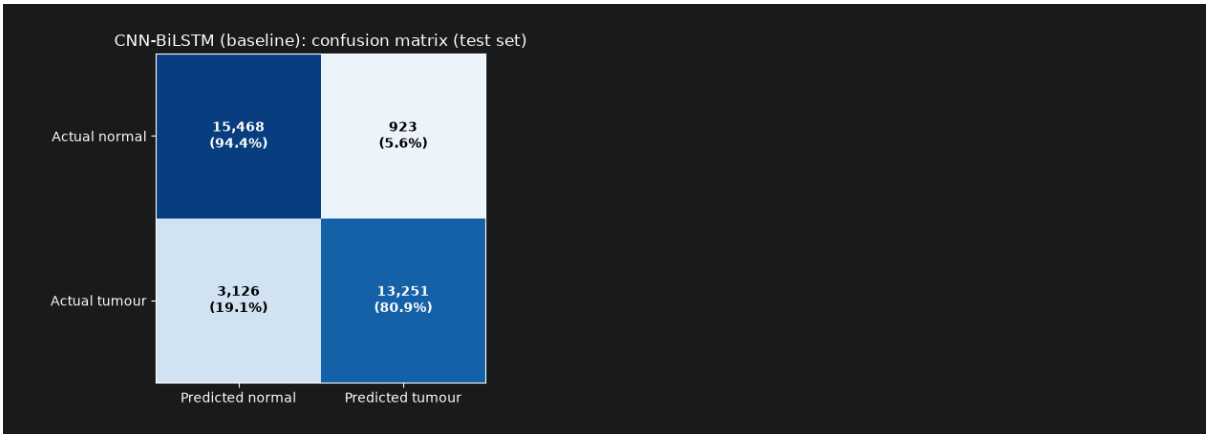


Figure 46

Code Snippet for ROC Curve of the Champion CNN-BiLSTM Model on the Test Set

```
fpr, tpr, _ = roc_curve(y_test, test_probs)
fig, ax = plt.subplots(figsize=(5.5, 5))
ax.plot(fpr, tpr, color="#2E6F9E", label=f"ROC curve (AUC = {m_test['auc']:.4f})")
ax.plot([0, 1], [0, 1], "--", color="grey", label="Random classifier")
ax.scatter(1 - m_test["specificity"], m_test["sensitivity"], s=60, zorder=5, color="#C1666B",
           label=f"Youden threshold = {thr_champ:.3f}")
ax.set(xlabel="False Positive Rate", ylabel="True Positive Rate",
       title=f"{champion_label}: ROC curve (test set)")
ax.legend(fontsize=8, loc="lower right")
plt.tight_layout()
plt.show()
```

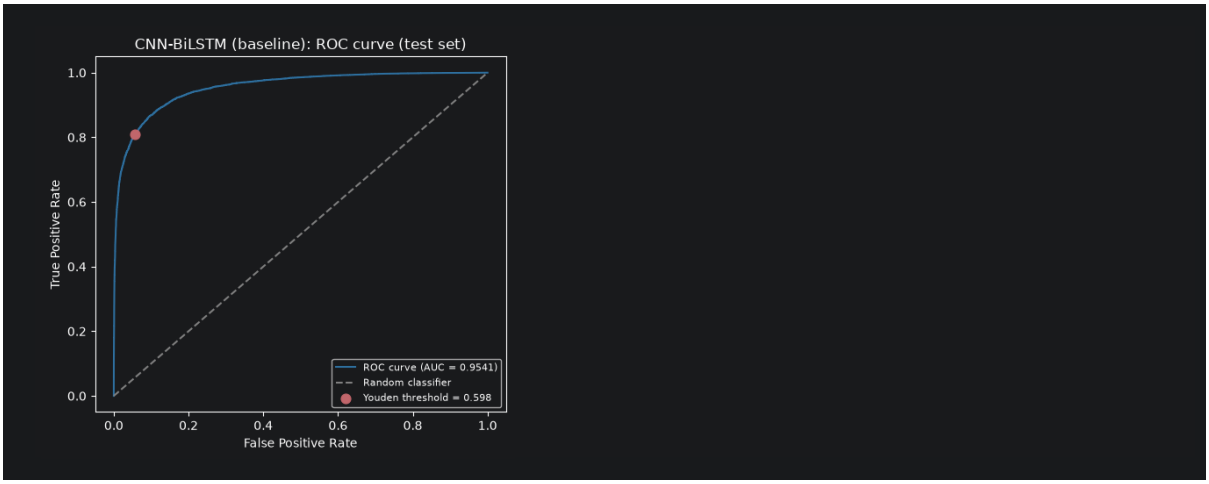
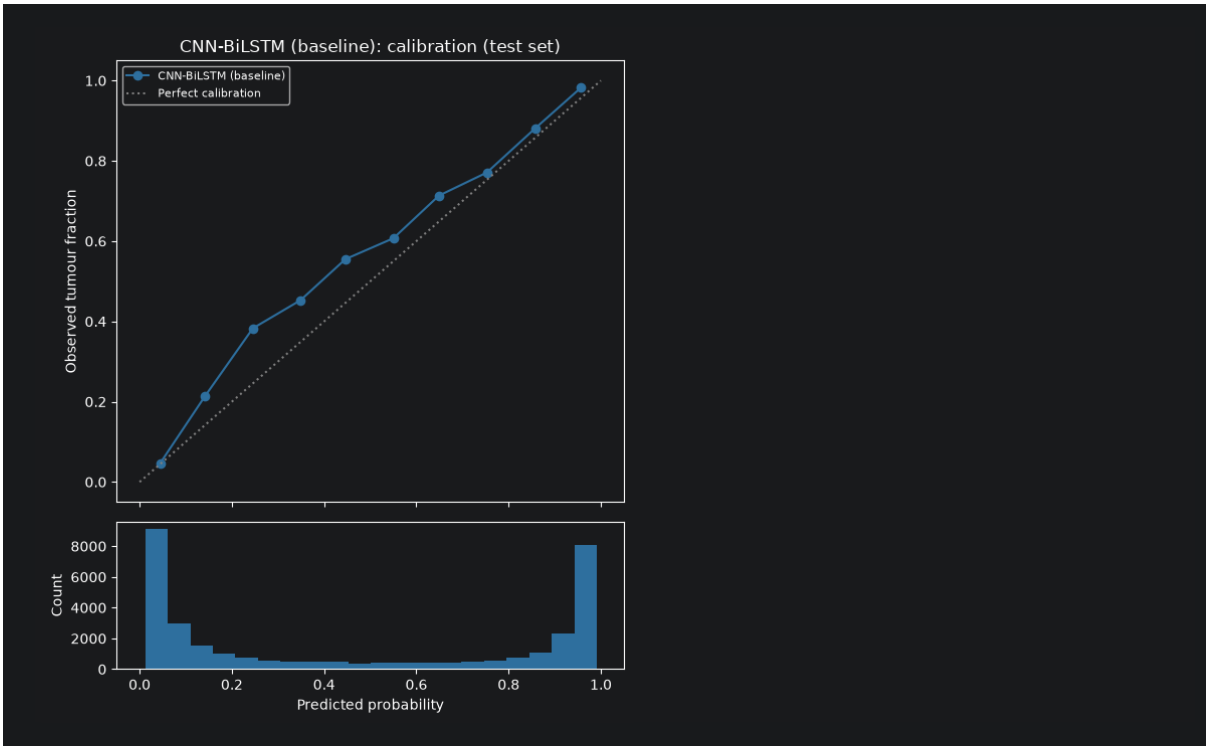


Figure 47

Code Snippet for Calibration Curve and Predicted Probability Distribution of Champion Model

```
frac_pos, mean_pred = calibration_curve(y_test, test_probs, n_bins=10)
fig, (ax, ax_hist) = plt.subplots(2, 1, figsize=(6, 7), sharex=True, gridspec_kw={"height_ratios": [3, 1]})
ax.plot(mean_pred, frac_pos, "o-", color="#2E6F9E", label=champion_label)
ax.plot([0, 1], [0, 1], ":", color="grey", label="Perfect calibration")
ax.set(ylabel="Observed tumour fraction", title=f"{champion_label}: calibration (test set)")
ax.legend(fontsize=8)
ax_hist.hist(test_probs, bins=20, color="#2E6F9E")
ax_hist.set(xlabel="Predicted probability", ylabel="Count")
plt.tight_layout()
plt.show()
```



Conclusion

In summary, this project explores the application of deep learning in histopathology with binary image classification. Inspired by the potential utilisation of medical image data to improve the efficiency and accuracy of clinical workflow, two deep learning models were developed to classify sentinel lymph node patches from the PatchCamelyon (PCam) dataset into normal and tumour tissue, namely the baseline convolutional neural network (CNN) model that is inspired by the architectural design of Simonyan and Zisserman (2015), as well as the CNN-bidirectional long short-term memory (CNN-BiLSTM) model that is less frequently explored in the current literature of deep learning in histopathology. After performing an initial exploratory data analysis (EDA) to identify patterns within the dataset and carrying out relevant data preprocessing steps to prepare the image patches, both models were trained, tuned using random search, and retrained with their best configurations. The champion model, which is the baseline CNN-BiLSTM model, was then selected using validation results before it was further evaluated on PCam's test split to assess its generalisability on unseen data.

Despite their relatively simple architectures, both trained models achieve satisfactory performance, where the champion model obtained a test accuracy of 87.64%, an ROC-AUC of 0.9541, and a negative log-likelihood of 0.2828, on par with the DenseNet from Veeling et al. (2018). Yet, the champion model does not reach the performance of advanced CNN models that implement mechanisms like skip connections, such as the Boosted EfficientNet-B3 from Wang et al. (2021) and the ResNet from Bütün et al. (2023), with accuracies of 97.96% and 98.60%. While these results are not directly comparable as both studies used modified versions of PCam, the adoption of advanced CNN models remains a promising direction for future work.

Overall, the strength of this project lies in its methodology. As the PCam dataset is sourced from only two medical centres, Tellez et al. (2019) outlined that models may learn the staining characteristics of the dataset instead of learning the morphological structure of tissues, which may impact the generalisability of models. Therefore, this project addresses the risk by incorporating multiple data augmentation techniques, such as D4 dihedral transformations to flip and rotate images, haematoxylin and eosin (H&E) stain jitter to vary staining intensity, and photometric jitter to alter brightness and contrast, so that variations can be introduced during model training. Additionally, the project extends its evaluation beyond accuracy to sensitivity and calibration to assess the suitability of trained models for clinical use. While the sensitivity of the champion model requires improvement, the project demonstrates the potential of deep learning models as tools to reduce pathologists' workload in classifying lymph node patches.

Limitations

Throughout the project, several limitations were encountered, including:

- **Stratified-sampled training data**

  The deep learning models were trained on a local machine with limited memory and computational power from the CPU and GPU. Therefore, due to hardware constraints, the PCam training split was sampled from 262,144 patches to 70,000 patches to ensure that the time taken for model training remained feasible. As only approximately 27% of the training data was used for training, this may limit the performance and generalisability of both the CNN and CNN-BiLSTM models.

- **Limited hyperparameter tuning budget**

  To ensure that the hyperparameter tuning process remained feasible, this project limited the number of search trials per model to only 20 random configurations due to time and resource constraints, where each trial was trained for only 6 epochs on a further reduced subset. As demonstrated by the optimised CNN-BiLSTM model, the limited number of trials may not accurately reflect the actual performance of models during full training, resulting in configurations that favour large learning steps and reduced regularisation that may lead to unstable learning and poorer model calibration.

- **Limited diversity of the dataset**

  The patches in the PCam dataset were extracted from the CAMELYON16 challenge dataset, which was collected from only two medical centres in the Netherlands. This may impact the performance of the trained models when patches are obtained from hospitals with varying staining protocols, scanner configurations and patient populations. Although steps like data augmentation are applied to mitigate this issue, the lack of diversity in the dataset limits the generalisability of the models, as the models may not learn additional useful patterns that may be present in sentinel lymph node images from other institutions.

Recommendations

Based on the limitations, several recommendations are outlined as directions for future work:

- **Train on the full PCam training set**

  Future researchers with access to more powerful computational resources, such as high-performance computing (HPC) systems or cloud-based GPUs, could train the deep learning models using all 262,144 patches from the training split, so that the models can be exposed to a wider variety of patterns that are present among sentinel lymph node tissues, which in turn improves the predictive capability and generalisability of the trained deep learning models.

- **Adopt a more efficient hyperparameter tuning strategy**

  Instead of using conventional hyperparameter tuning strategies like grid search and random search, future work could apply more advanced methods, such as Hyperband, which is able to terminate configurations with poor performance earlier and allocate additional computational resources to the more promising ones. By incorporating more powerful computational resources with these advanced techniques during the tuning process, future work could evaluate a larger number of configurations across different training budgets so that the best hyperparameter configuration for both the CNN and CNN-BiLSTM models can be identified.

- **Validate the models on multi-centre data**

  To measure the robustness and quality of the predictions produced by the trained models, future work could perform evaluation using multiple datasets, such as CAMELYON17 that contains the whole-slide imaging (WSI) data of sentinel lymph node tissues from multiple medical centres. By performing predictions on patches with different scanning and staining conditions, the evaluation metrics obtained from these datasets could further enhance the validity of the models when applied in clinical settings.

References

Aguilar-Ruiz, J., & Michalak, M. (2024). Classification performance assessment for imbalanced multiclass data. *Scientific Reports, 14*, 10759.

Alzubaidi, L., Zhang, J., Humaidi, A., Al-Dujaili, A., Duan, Y., Al-Shamma, O., . . . Farhan, L. (2021). Review of deep learning: Concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data, 8*, 53.

Ashraf, M., Robles, W., Kim, M., Ko, Y., & Yi, M. (2022). A loss-based patch label denoising method for improving whole-slide image analysis using a convolutional neural network. *Scientific Reports, 12*, 1392.

Atallah, N., Toss, M., Verrill, C., Salto-Tellez, M., Snead, D., & Rakha, E. (2022). Potential quality pitfalls of digitalized whole slide image of breast pathology in routine practice. *Modern Pathology, 35*(7), 903-910.

Aydin, F., Ozcelik , K., Tunc, E., Aslan, F., Kulturoglu, M., Sagdic, M., . . . Dogan, L. (2026). Omission of frozen section in sentinel lymph node biopsy after neoadjuvant chemotherapy in cN0 HER2-positive and TNBC. *Frontiers in Oncology, 16*, 1-6.

Balkenende, L., Teuwen, J., & Mann, R. (2022). Application of deep learning in breast cancer imaging. *Seminars in Nuclear Medicine, 52*(5), 584-596.

Barnachea, D., & Lee, B. (2026). Sentinel lymph node biopsy in melanoma: Pathologic evaluation and diagnostic considerations. *Journal of Pathology and Translational Medicine, 60*(5), 493-501.

Bjorck, J., Gomes, C., Selman, B., & Weinberger, K. (2018). Understanding batch normalization. *arXiv*, 1-24.

Bojnordi, A., Nezhad, M., Bagheri, R., Bazrafshan, M., & Sohrabi, B. (2025). Identifying, ranking and analyzing obstacles to big data analytics implementation in the healthcare industry using an ISM approach. *Discover Health Systems, 4*(32).

Bray, F., Laversanne, M., Sung, H., Ferlay, J., Siegel, R., Soerjomataram, I., & Jemal, A. (2024). Global cancer statistics 2022: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries. *CA: A Cancer Journal for Clinicians, 74*(3), 229-263.

Bronstein, M., Bruna, J., Cohen, T., & Veličković, P. (2021). Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. *arXiv*, 1-156.

Burguin, A., Diorio, C., & Durocher, F. (2021). Breast cancer treatments: Updates and new challenges. *Journal of Personalized Medicine, 11*(8), 808.

Butt, N., Sarwat, D., Noya, I., Tutusaus, K., Samee, N., & Ashraf, I. (2026). Benchmarking multiple instance learning architectures from patches to pathology for prostate cancer detection and grading using attention-based weak supervision. *Scientific Reports, 16*(1), 11535.

Bütün, E., Uçan, M., & Kaya, M. (2023). Automatic detection of cancer metastasis in lymph node using deep learning. *Biomedical Signal Processing and Control, 82*, 104564.

Cabot, J., & Ross, E. (2023). Evaluating prediction model performance. *Surgery, 174*(3), 723-726.

Cao, H., & Ma, D. (2025). Digital Health: Transforming Healthcare Through Technology. In S. Detscher, & M. Hepp, *Praxishandbuch Digitales Management* (pp. 1-17). Wiesbaden: Springer Gabler.

Carvalho, J., Santinha, J., Miladinović, Đ., Cotrini, C., & Buhmann, J. (2022). Holistic modeling in medical image segmentation using spatial recurrence. *Proceedings of Machine Learning Research*, *172*, pp. 1-20.

Chang, C.-P., Hsu, C.-Y., Wang, H., Feng, P.-C., & Liang, W.-Y. (2025). Detection of metastatic breast carcinoma in sentinel lymph node frozen sections using an artificial intelligence-assisted system. *Pathology, Research and Practice, 267*, 155836.

Connal, S., Cameron, J., Sala, A., Brennan, P., Palmer, D., Palmer, J., . . . Baker, M. (2023). Liquid biopsies: The future of cancer early detection. *Journal of Translational Medicine, 21*(1), 118.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., . . . Houlsby, N. (2021). An image is worth 16x16 words: Transformers for image recognition at scale. *International Conference on Learning Representations 2021*, (pp. 1-22).

Edwards, M., Elder, K., Rose, A., Tan, E., Park, A., Nickson, C., & Mann, G. (2025). Survival benefits and less intensive treatment for women with early-stage breast cancer diagnosed while participating in population-based screening. *Annals of Surgical Oncology, 32*(11), 8100-8110.

Evans, H., & Snead, D. (2024). Understanding the errors made by artificial intelligence algorithms in histopathology in terms of patient impact. *NPJ Digital Medicine, 7*(1), 89.

Faghani, S., Moassefi, M., Rouzrokh, P., Khosravi, B., Baffour, F., Ringler, M., & Erickson, B. (2023). Quantifying uncertainty in deep learning of radiologic images. *Radiology, 308*(2), e222217.

Feng, J., Li, C., Jiang, X., Zhou, L., Jiang, Y., & Deng, B. (2026). Raman and surface-enhanced Raman spectroscopy for intraoperative cancer diagnostics: Sentinel lymph node biopsy, biomarkers and translational challenges. *Frontiers in Bioengineering and Biotechnology, 14*, 1-23.

Ghosh, K., Bellinger, C., Corizzo, R., Branco, P., Krawczyk, B., & Japkowicz, N. (2024). The class imbalance problem in deep learning. *Machine Learning, 113*, 4845-4901.

Grand Challenge. (2026). *PatchCamelyon*. Retrieved July 10, 2026, from Grand Challenge: https://patchcamelyon.grand-challenge.org/

Hanna, T., King, W., Thibodeau, S., Jalink, M., Paulin, G., Harvey-Jones, E., . . . Aggarwal, A. (2020). Mortality due to cancer treatment delay: Systematic review and meta-analysis. *BMJ (Clinical research ed.), 371*, m4087.

Hassanzad, M., & Hajian-Tilaki, K. (2024). Methods of determining optimal cut-point of diagnostic biomarkers with application of clinical data in ROC analysis: An update review. *BMC Medical Research Methodology, 24*, 84.

Ilse, M., Tomczak, J., & Welling, M. (2018). Attention-based deep multiple instance learning. *arXiv*, 1-16.

Islam, M., Hoque, M., Ullah, M., Islam, M., Nishu, N., & Islam, M. (2024). CNN-based deep learning approach for classification of invasive ductal and metastasis types of breast carcinoma. *Cancer Medicine, 13*(16), e70069.

Jeczmionek, E., & Kowalski, P. (2021). Flattening layer pruning in convolutional neural networks. *Symmetry, 13*(7), 1147.

Kim, H., Jeong, W., Hwang, Y., Hwang, G., Kim, K., & Ko, Y. (2026). Comprehensive and reproducible grading of the Updated Sydney System in gastric biopsies using artificial intelligence: Multi-pathologist validation and a reader study. *Histopathology*.

Kotsifaki, A., Masoura, C.-R., Limogianni, G., Kalouda, G., Stathaki, M., & Armakolas, A. (2026). Non-invasive diagnosis of early breast cancer: Current and emerging liquid biopsy biomarkers. *Cancers, 18*(14), 2344.

Koyun, O., Guo, Y., Lu, H., Demir, M., Alili, A., Rahoui, N., . . . Gurcan, M. (2026). Slide selection introduces sampling-induced prediction uncertainty in digital pathology AI: Evidence from multi-slide breast cancer cohorts. *Cancers, 18*(16), 2540.

Leo, P., Elliott, R., Shih, N., Gupta, S., Feldman, M., & Madabhushi, A. (2018). Stable and discriminating features are predictive of cancer presence and Gleason grade in radical prostatectomy specimens: A multi-site study. *Scientific Reports, 8*, 14918.

Liang, Y., Niu, C., Yan, P., & Wang, G. (2024). Flipover outperforms dropout in deep learning. *Visual Computing for Industry, Biomedicine, and Art, 7*(1), 4.

Liu, B., Ayed, I., Galdran, A., & Dolz, J. (2022). The devil is in the margin: Margin-based label smoothing for network calibration. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 80-88). IEEE.

Liu, H., Zhang, X., Zhang, S., Wang, X., & Yu, S. (2021). Factors associated with bone metastasis in breast cancer: A systematic review and meta-analysis. *Annals of Palliative Medicine, 10*(4), 4435-4452.

Mahdi, O., & Nassif, A. (2022). Transformation invariant cancerous tissue classification using spatially transformed DenseNet. *2022 Advances in Science and Engineering Technology International Conferences* (pp. 1-6). IEEE.

Mahmoud, H., Omer, O., Ragab, S., Esmaiel, H., & Abdel-Nasser, M. (2024). Classifying melanoma in ISIC dermoscopic images using efficient convolutional neural networks and deep transfer learning. *Traitement du Signal, 41*(2), 679-691.

Malashin, I., Tynchenko, V., Gantimurov, A., Nelyub, V., & Borodulin, A. (2024). Applications of long short-term memory (LSTM) networks in polymeric sciences: A review. *Polymers, 16*(18), 2607.

Mao, Y., Gupta, V., Wang, K., Liao, W.-k., Choudhary, A., & Agrawal, A. (2022). To shuffle or not to shuffle: Mini-batch shuffling strategies for multi-class imbalanced classification. *2022 International Conference on Computational Science and Computational Intelligence (CSCI)* (pp. 298-301). Las Vegas, NV, USA: IEEE.

Nallapu, V., & Roychowdhury, M. (2026). Hidden symmetries of the classical 3×3 magic square. *ISQGD Recreational Mathematics Journal, 1*(1), a005.

Piso, A., Teer, E., Ramashala, U., Mercier, A., Engelbrecht, A.-M., & van den Bout, I. (2026). Advances in biological breast cancer research and cancer care in Africa. *Preprints*.

Quirin, T., Féry, C., Vogel, D., Vergne, C., Sarracanie, M., Salameh, N., . . . Pascal, J. (2021). Towards tracking of deep brain stimulation electrodes using an integrated magnetometer. *Sensors, 21*(8), 2670.

Reininghaus, M. (2023). cnpy++: A C++17 library for reading and writing .npy/.npz files. *SoftwareX, 21*, 101324.

Ruifrok, A., & Johnston, D. (2001). Quantification of histochemical staining by color deconvolution. *Analytical and Quantitative Cytology and Histology, 23*(4), 291-299.

Salama, D., Allen, B., Frija, G., Rockall, A., Biljacic, B., Mikhail, M., . . . Paez, D. (2026). From struggle to strength: transforming breast cancer screening for women in low- and middle-income countries. *British Journal of Radiology, 99*(1180), 603-608.

Salehin, I., & Kang, D.-K. (2023). A review on dropout regularization Approaches for deep neural networks within the scholarly domain. *Electronics, 12*(14), 3106.

Schlett, T., Rathgeb, C., Tapia, J., & Busch, C. (2024). Double trouble? Impact and detection of duplicates in face image datasets. *Proceedings of the 13th International Conference on Pattern Recognition Applications and Methods* (pp. 801-808). Science and Technology Publications.

Sclocchi, A., & Wyart, M. (2024). On the different regimes of stochastic gradient descent. *Proceedings of the National Academy of Sciences, 121*(9), e2316301121.

Sharma, D., & Saripilli, R. (2025). Recent strategies in diagnosis, screening, prevention, and treatment of breast cancer in young women. *Discover Oncology, 16*, 1532.

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *arXiv*, 1-14.

Srikantamurthy, M., Rallabandi, V., Dudekula, D., Natarajan, S., & Park, J. (2023). Classification of benign and malignant subtypes of breast cancer histopathology imaging using hybrid CNN-LSTM based transfer learning. *BMC Medical Imaging, 23*(1), 19.

Srinidhi, C., Ciga, O., & Martel, A. (2021). Deep neural network models for computational histopathology: A survey. *Medical Image Analysis, 67*, 101813.

Tauqeer, A., Asif, A., & Sadeghi-Naini, A. (2025). Detection, localization, and staging of breast cancer lymph node metastasis in digital pathology whole slide images using selective neighborhood attention-based deep learning. *Scientific Reports, 15*, 37847.

Tavolara, T., Gurcan, M., & Niazi, M. (2022). Contrastive multiple instance learning: An unsupervised framework for learning slide-level representations of whole slide histopathology images without labels. *Cancers, 14*(23), 5778.

Tellez, D., Litjens, G., Bándi, P., Bulten, W., Bokhorst, J.-M., Ciompi, F., & van der Laak, J. (2019). Quantifying the effects of data augmentation and stain color normalization in convolutional neural networks for computational pathology. *Medical Image Analysis, 58*, 101544.

Terven, J., Cordova-Esparza, D.-M., Romero-González, J.-A., Ramírez-Pedraza, A., & Chávez-Urbiola, E. (2025). A comprehensive survey of loss functions and metrics in deep learning. *Artificial Intelligence Review, 58*, 195.

Udeh, C., Orieno, O., Daraojimba, O., Ndubuisi, N., & Oriekhoe, O. (2024). Big data analytics: A review of its transformative role in modern business intelligence. *Computer Science & IT Research Journal, 5*(1), 219-236.

van Dooijeweert, C., Flach, R., ter Hoeve, N., Vreuls, C., Goldschmeding, R., Freund, J., . . . van Diest, P. (2024). Clinical implementation of artificial-intelligence-assisted detection of breast cancer metastases in sentinel lymph nodes: The CONFIDENT-B single-center, non-randomized clinical trial. *Nature Cancer, 5*(8), 1195-1205.

Veeling, B., Linmans, J., Winkens, J., Cohen, T., & Welling, M. (2018). Rotation Equivariant CNNs for Digital Pathology. In A. Frangi, J. Schnabel, C. Davatzikos, C. Alberola-López, & G. Fichtinger, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018* (Vol. 11071, pp. 210-218). Springer, Cham.

Vincent, A., & Jidesh, P. (2023). An improved hyperparameter optimization framework for AutoML systems using evolutionary algorithms. *Scientific Reports, 13*, 4737.

Wang, J., Liu, Q., Xie, H., Yang, Z., & Zhou, H. (2021). Boosted EfficientNet: Detection of lymph node metastases in breast cancer using convolutional neural networks. *Cancers, 13*(4), 661.

Wang, X., Girshick, R., Gupta, A., & He, K. (2018). Non-local neural networks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 7794-7803). Salt Lake City, UT, USA: IEEE.

Weaver, D., Krag, D., Manna, E., Ashikaga, T., Harlow, S., & Bauer, K. (2003). Comparison of pathologist-detected and automated computer-assisted image analysis detected sentinel lymph node micrometastases in breast cancer. *Modern Pathology, 16*, 1159-1163.

Windsor, G., Bai, H., Lourenco, A., & Jiao, Z. (2023). Application of artificial intelligence in predicting lymph node metastasis in breast cancer. *Frontiers in Radiology, 3*.

Wojciuk, M., Swiderska-Chadaj, Z., Siwek, K., & Gertych, A. (2024). Improving classification accuracy of fine-tuned CNN models: Impact of hyperparameter optimization. *Heliyon, 10*(5), e26586.

World Health Organization. (2026, July 3). *Breast cancer*. Retrieved from World Health Organization: https://www.who.int/news-room/fact-sheets/detail/breast-cancer

Xu, F., Zhu, C., Tang, W., Wang, Y., Zhang, Y., Li, J., . . . Jin, M. (2021). Predicting axillary lymph node metastasis in early breast cancer using deep learning on primary tumor biopsy slides. *Frontiers in Oncology, 11*, 759007.

Yang, Q., Liu, C., Wang, Y., Dong, G., & Sun, J. (2025). Construction of risk prediction model of sentinel lymph node metastasis in breast cancer patients based on machine learning algorithm. *Discover Oncology, 16*(1), 704.

Zhou, P., Xie, X., Lin, Z., & Yan, S. (2024). Towards understanding convergence and generalization of AdamW. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 46*(9), 6486-6493.